Playwright Notes

Part 1: Getting Started & Features (commands, locators, assertions, agents, config, fixtures, environments, CI, sharding, timeouts, best practices) · Part 2: 35 Guides · TypeScript · based on playwright.dev, with lead-level project use and interview answers

Index

Part 1 · Playwright Features		Part 2 · Playwright Guides	
	Quick Summary		Guides overview (index of G1–G35)
1	Installation & Project Structure	**G1**	Library
2	All Commands in One Place	**G2**	Accessibility testing
3	Writing Tests	**G3**	Actions
4	Locators in One Place	**G4**	Assertions
5	Assertions in One Place	**G5**	API testing
6	Generating Tests (Codegen), Running & Debugging	**G6**	Authentication
7	CI Setup (GitHub Actions)	**G7**	Auto-waiting
8	Real-Time Usage & Interview Questions	**G8**	Browsers
9	Playwright Test Agents: Planner → Generator → Healer	**G9**	Chrome extensions
9B	Instructing the Agents: Context Files + Prompts	**G10**	Clock
10	Annotations: Control Which Tests Run and How	**G11**	Component testing
11	Command Line: More Useful Options	**G12**	Debugging tests
12	Configuration: Useful Options, Global Setup & Teardown	**G13**	Dialogs
13	Configuration "use": Options & Scope (Global → Project → File → Describe)	**G14**	Downloads
14	Emulation: Mobiles & Tablets	**G15**	Evaluating JavaScript
15	Fixtures: Role-Based Users & Lead-Level Fixtures	**G16**	Events
16	Global Setup & Teardown: What Teams Use in Real Projects	**G17**	Extensibility
17	Parallelism: Run Faster Without Flakiness	**G18**	Frames
18	Parameterized (Data-Driven) Tests	**G19**	Handles
19	Projects & Multiple Environments (Local and CI)	**G20**	Isolation
20	Test Filtering: Run the Right Tests at the Right Time	**G21**	Locators
21	Reporters: Who Needs Which Report	**G22**	Mock APIs
22	Retries & Flaky Tests (Local vs CI)	**G23**	Mock browser APIs
23	Sharding with GitHub Actions & Merging Reports	**G24**	Navigations
24	Timeouts: Every Level Explained	**G25**	Network
25	UI Mode: The Daily Development & Debugging Cockpit	**G26**	Other locators
26	webServer: Start the App Before the Tests	**G27**	Pages
27	Best Practices: Lead & Architect Playbook	**G28**	Page object models
		G29	Screenshots
		G30	Service Workers
		G31	Snapshot testing
		G32	Test generator
		G33	Touch events (legacy)
		G34	Trace viewer
		G35	Videos

Click any entry to jump to it.

Quick Summary

Topic	Remember this
Install	`npm init playwright@latest` → creates `playwright.config.ts`, `tests/example.spec.ts`, `package.json`. Node 22/24/26.
Write	`test('name', async ({ page }) => { ... })` · locator → action → `await expect(...)`. Each test gets a fresh **BrowserContext** (isolated).
Generate	`npx playwright codegen <url>`: records clicks into code, prefers role/text/test-id locators.
Run	`npx playwright test` (+ `--ui`, `--headed`, `--project`, `-g`, `--last-failed`)
Debug	`--debug` (Inspector), `--ui` (UI mode, time travel), `await page.pause()`, VS Code extension
Report / Trace	`npx playwright show-report` · config `trace: 'on-first-retry'` · `npx playwright show-trace trace.zip`
CI	`npm ci` → `npx playwright install --with-deps` → `npx playwright test` → upload `playwright-report/` artifact
Locators	getByRole › getByLabel › getByPlaceholder › getByText › getByAltText › getByTitle › getByTestId › CSS/XPath (last resort)
Assertions	`await expect(locator).toBeVisible()` = **auto-retrying** (waits up to 5 s) · `expect(value).toBe()` = instant
Agents	`npx playwright init-agents --loop=vscode` · Planner (seed → `specs/*.md`) → Generator (→ `tests/*.spec.ts`) → Healer (fixes failing tests)
Annotations	`only` · `skip` · `fixme` · `fail` · `slow` · tags `{ tag: '@smoke' }` → `--grep @smoke` · `annotation: { type: 'issue', … }`
Config & use	Setup project + `storageState` for login once · `use` scope: global → project → file → describe (closest wins)
Devices & fixtures	Devices = projects (iPhone/iPad → WebKit, Pixel/Galaxy → Chromium) · fixtures: roles via storageState, test data, auto, worker
Part 2: Guides	35 guides (G1–G35): actions, auto-waiting, API testing, auth, frames, dialogs, downloads, network & mocking, clock, POM, screenshots, aria snapshots…: methods + syntax + project use
Scale & stability	fullyParallel + workers · data-driven loops · TEST_ENV + GitHub Environments · tags for PR/nightly/prod · blob → merge-reports · retries 0 local / 2 CI · timeouts 30 s test, 5 s expect

🧠 Lifecycle: **"Install → Write (or Codegen) → Run → Debug → Report → Trace → CI"** · Golden rule: **"Find by what the USER sees, assert with await, let Playwright wait."**

1 · Installation & Project Structure

```
npm init playwright@latest          # or: yarn create playwright / pnpm create playwright
# Prompts: TypeScript or JavaScript · tests folder name · add GitHub Actions workflow · install browsers
```

Created file / folder	Purpose
`playwright.config.ts`	Central config: browsers (projects), retries, workers, reporter, baseURL, trace
`tests/example.spec.ts`	Sample test (folder is `e2e/` if `tests/` already exists)
`package.json` + lock file	Dependency `@playwright/test`
`.github/workflows/playwright.yml`	Optional CI workflow (if chosen)
`playwright-report/`, `test-results/`	Created after a run: HTML report; traces, screenshots, videos

Requirements: Node.js 22.x / 24.x / 26.x · Windows 11+ (or WSL), macOS 14+, Debian 12/13, Ubuntu 22.04/24.04/26.04.

The config file explained (default generated shape):

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './tests',                       // where tests live
  fullyParallel: true,                      // run tests inside files in parallel too
  forbidOnly: !!process.env.CI,             // fail CI if someone left test.only
  retries: process.env.CI ? 2 : 0,          // retry flaky tests only on CI
  workers: process.env.CI ? 1 : undefined,  // CI: 1 worker; local: half the CPU cores
  reporter: 'html',                         // HTML report in playwright-report/
  use: {
    baseURL: 'http://localhost:3000',       // then page.goto('/login')
    trace: 'on-first-retry',                // record trace when a failed test is retried
  },
  projects: [                               // one project = one browser/device setup
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
    { name: 'firefox',  use: { ...devices['Desktop Firefox'] } },
    { name: 'webkit',   use: { ...devices['Desktop Safari'] } },
  ],
});
```

> 🧠 CI config = **"2 retries, 1 worker, no .only, trace on first retry"**.

2 · All Commands in One Place

Command	What it does
Install / update	
`npm init playwright@latest`	Create a new Playwright project
`npx playwright install` · `install chromium` · `install --with-deps`	All browsers · one browser · browsers + OS dependencies (CI/Linux)
`npm install -D @playwright/test@latest`	Update Playwright (then run install again)
`npx playwright --version`	Show installed version
Run	
`npx playwright test`	All tests, all projects, headless
`npx playwright test --headed`	Visible browser
`npx playwright test --ui`	**UI mode**: watch, time-travel, pick locators
`npx playwright test --project=chromium`	One browser project (repeat flag for more)
`npx playwright test login.spec.ts` · `tests/payments/`	One file · one folder
`npx playwright test login payment`	Files whose names contain these words
`npx playwright test -g "transfer funds"`	Tests whose title matches (`--grep`)
`npx playwright test --grep @smoke` / `--grep-invert @slow`	Run / exclude by tag
`npx playwright test --last-failed`	Re-run only the failures from the previous run
`--workers=4` · `--retries=2` · `--repeat-each=5` · `-x`	Parallelism · retries · repeat (flakiness) · stop at first failure (`--max-failures=N`)
`--shard=1/4`	Run part 1 of 4 (split across CI machines)
`--reporter=line` (list, dot, html, json, junit, blob)	Choose output format
`--update-snapshots` · `--list`	Accept new screenshot baselines · list tests without running
Debug	
`npx playwright test --debug`	Playwright **Inspector**: step through, headed
`npx playwright test login.spec.ts:10 --debug`	Debug the test at line 10
`await page.pause()` (in code)	Pause at this line and open the Inspector
Report & trace	
`npx playwright show-report` [folder]	Open the HTML report (needs a local server; don't double-click the file)
`npx playwright test --trace on`	Force traces for this run
`npx playwright show-trace trace.zip`	Open a trace file (or drop it on trace.playwright.dev)
`npx playwright merge-reports --reporter html ./blob-reports`	Merge sharded CI results into one report
Codegen	
`npx playwright codegen https://app.example.com`	Record actions → test code
`codegen --device="iPhone 13"` · `--viewport-size="1280,720"` · `--color-scheme=dark`	Emulate device, screen size, theme (also `--lang`, `--timezone`, `--geolocation`)
`codegen --save-storage=auth.json` / `--load-storage=auth.json`	Save / reuse a logged-in session

🧠 **"test runs, show shows, codegen records, install installs"**. Debug trio: **--debug** (Inspector) · **--ui** (time travel) · **page.pause()** (stop here).

3 · Writing Tests

```
import { test, expect } from '@playwright/test';

test.describe('Login', () => {                              // group of tests
  test.beforeEach(async ({ page }) => {
    await page.goto('/login');                               // uses baseURL from config
  });

  test('valid user lands on dashboard', { tag: '@smoke' }, async ({ page }) => {
    await page.getByLabel('Customer ID').fill('C123456');     // 1. LOCATE + ACT
    await page.getByLabel('Password').fill(process.env.TEST_PWD!);
    await page.getByRole('button', { name: 'Log in' }).click();
    await expect(page).toHaveURL(/dashboard/);                // 2. ASSERT (auto-waits)
    await expect(page.getByRole('heading', { name: 'Accounts' })).toBeVisible();
  });
});
```

Concept	Meaning
Fixtures (`{ page }`)	Ready-made objects injected into tests: `page`, `context`, `browser`, `request` (API), `browserName`. Custom ones via `test.extend()`.
Test isolation	Every test gets its own **BrowserContext** (like a new incognito profile): no shared cookies/storage.
Hooks	`test.describe` groups · `beforeEach`/`afterEach` per test · `beforeAll`/`afterAll` once per worker
Auto-waiting	Actions wait until the element is visible, stable, enabled and receives events: **no manual sleeps**
Annotations	`test.only`, `test.skip`, `test.fixme`, `test.fail`, `test.slow`, tags `{ tag: '@smoke' }`
Steps	`await test.step('Fill transfer form', async () => { ... })`: readable report sections

Actions (on a locator):

Action	Use	Action	Use
`goto(url)` (page)	Navigate	`check()` / `uncheck()`	Checkbox / radio
`click()` / `dblclick()`	Click	`selectOption('INR')`	Dropdown
`fill('text')`	Type into input (clears first)	`setInputFiles('a.pdf')`	Upload
`pressSequentially('abc')`	Type key by key	`hover()` / `focus()`	Mouse over / focus
`press('Enter')`	Single key	`dragTo(target)`	Drag and drop

4 · Locators in One Place

#	Locator	Finds by	Example
1	**`getByRole()`**	ARIA role + accessible name (how users & screen readers see it)	`getByRole('button', { name: 'Transfer' })`
2	`getByLabel()`	Form control by its label	`getByLabel('Amount')`
3	`getByPlaceholder()`	Input placeholder	`getByPlaceholder('Enter IFSC')`
4	`getByText()`	Visible text (non-interactive elements)	`getByText('Payment successful')`
5	`getByAltText()`	Image alt text	`getByAltText('Bank logo')`
6	`getByTitle()`	`title` attribute	`getByTitle('Account settings')`
7	`getByTestId()`	`data-testid` (change via `testIdAttribute`)	`getByTestId('balance-amount')`
8	`locator()` CSS / XPath	Last resort; avoid long DOM paths	`locator('#otp')`, `locator('//table//tr[2]')`

> 🧠 Priority: **"Really Lazy People Take A Tiny Test"** = Role, Label, Placeholder, Text, Alt, Title, TestId (then CSS/XPath).

Refine a locator	Example
Filter by text (`hasText` / `hasNotText`)	`page.getByRole('row').filter({ hasText: 'Ravi' })`
Filter by child (`has` / `hasNot`)	`rows.filter({ has: page.getByRole('button', { name: 'Pay' }) })`
Only visible	`locator.filter({ visible: true })`
Chaining (search inside)	`page.getByTestId('payee-list').getByRole('button', { name: 'Delete' })`
Position (discouraged)	`.first()`, `.last()`, `.nth(2)`
Both / either	`a.and(b)` · `a.or(b)` (add `.first()` if both may show)
iframe	`page.frameLocator('#payment-frame').getByLabel('Card number')`
Shadow DOM	Pierced automatically (not by XPath; not closed shadow roots)
Many elements	`await rows.count()`, `await rows.all()`

Strictness: a single-element action (`click`, `fill`) **throws if the locator matches more than one element**. Fix by making the locator more specific, not by adding `.first()` everywhere.

Real-time tips: ask developers to add `data-testid` where roles/text are ambiguous; pick locators in `--ui` mode or codegen's "Pick locator"; locators are **lazy** (found again at each action), so they survive re-renders.

5 · Assertions in One Place

Two kinds: auto-retrying (web-first) assertions are `await`ed and **retry until true or timeout (5 s default)**; **non-retrying** ones check a plain value once.

Auto-retrying (use `await`)	Checks	Auto-retrying	Checks
`toBeVisible()` / `toBeHidden()`	Shown / not shown	`toHaveText()`	Exact text (string/regex)
`toBeEnabled()` / `toBeDisabled()`	Clickable or not	`toContainText()`	Contains text
`toBeChecked()`	Checkbox ticked	`toHaveValue()` / `toHaveValues()`	Input / multi-select value
`toBeEditable()` · `toBeFocused()`	Editable · has focus	`toHaveCount()`	Number of matches
`toBeAttached()` · `toBeEmpty()`	In DOM · no content	`toHaveAttribute()` · `toHaveId()`	Attribute · id
`toBeInViewport()`	Inside the screen area	`toHaveClass()` · `toContainClass()` · `toHaveCSS()`	Classes · CSS value
`toHaveRole()` · `toHaveAccessibleName()` · `toHaveAccessibleDescription()`	Accessibility	`toHaveJSProperty()`	JS property
`expect(page).toHaveTitle()` · `toHaveURL()`	Page title · URL	`toHaveScreenshot()`	Visual comparison (element/page)
`toMatchAriaSnapshot()`	Accessibility-tree snapshot	`expect(response).toBeOK()`	API status 2xx

Non-retrying (plain values)	Examples
Equality	`toBe()`, `toEqual()` (deep), `toStrictEqual()`, `toMatchObject()`
Truthiness	`toBeTruthy()`, `toBeFalsy()`, `toBeNull()`, `toBeUndefined()`, `toBeDefined()`, `toBeNaN()`
Numbers	`toBeGreaterThan()`, `…OrEqual()`, `toBeLessThan()`, `toBeCloseTo()`
Strings / arrays / objects	`toContain()`, `toContainEqual()`, `toHaveLength()`, `toMatch(/regex/)`, `toHaveProperty()`, `toBeInstanceOf()`, `toThrow()`
Asymmetric matchers	`expect.any(Number)`, `expect.anything()`, `expect.objectContaining()`, `expect.arrayContaining()`, `expect.stringContaining()`, `expect.stringMatching()`, `expect.closeTo()`

Special forms	Example / use
Negate	`await expect(errorBanner).not.toBeVisible()`
Soft (keep going, fail at end)	`await expect.soft(page.getByTestId('fee')).toHaveText('₹0')`
Custom message	`await expect(balance, 'balance after transfer').toHaveText('₹4,500')`
Poll a function (e.g. API)	`await expect.poll(async () => (await api.get('/status')).status()).toBe(200)`
Retry a block	`await expect(async () => { ... }).toPass({ timeout: 30_000 })`
Timeouts	Config `expect: { timeout: 10_000 }` · per call `toBeVisible({ timeout: 15_000 })`

> **Classic mistake:** `expect(await locator.textContent()).toBe('Paid')` checks **once, immediately** (flaky). Use `await expect(locator).toHaveText('Paid')`, which **waits and retries**.

> 🧠 **"await expect(LOCATOR) waits; expect(VALUE) doesn't."** · "To-Be a state, To-Have a property."

6 · Generating Tests (Codegen), Running & Debugging

Tool	How	Real-time use
Codegen	`npx playwright codegen <url>`: a browser + Inspector open; your clicks/typing become code. Prefers **role, text, test-id** locators and makes them unique. Toolbar: **assert visibility / text / value**, **Pick locator**, copy code.	Fast first draft and locator discovery; then clean up into Page Objects (don't commit raw recordings)
Codegen emulation	Device, viewport, colour scheme, geolocation, language, timezone; `--save-storage` keeps login state	Mobile flows; record once logged in
UI mode (`--ui`)	Test list, watch mode, **time-travel** through each action with DOM snapshots, network, console, locator picker	Daily development and fixing flaky tests
Inspector (`--debug`)	Headed run paused at the start; step over actions, see the locator highlighted, edit locators live	Step-by-step debugging of one test
`page.pause()`	Stops the test at that line and opens the Inspector	Investigate the state mid-test
VS Code extension	Run/debug from the gutter, breakpoints, pick locator, record new test	Most convenient in the IDE
HTML report	`show-report`: pass/fail per browser, errors, screenshots, videos, traces, steps	Share results; open traces of failures

Trace Viewer

Record	Open	What you see
Config `use: { trace: 'on-first-retry' }` (+ `retries` on CI) Modes: `'off'`, `'on'`, `'retain-on-failure'`, `'on-first-retry'`, `'on-all-retries'` Or: `npx playwright test --trace on`	HTML report → trace icon / "Traces" tab `npx playwright show-trace test-results/…/trace.zip` or drag the zip onto **trace.playwright.dev** (runs in your browser)	Timeline of every action · **before/after DOM snapshots** (inspectable) · locator used · log · source line · **network** calls · **console** · errors

> 🧠 Trace = **"a video you can inspect"**: every action, the page before/after, network and console. Record **on-first-retry** to keep CI fast.

7 · CI Setup (GitHub Actions)

```yaml
# .github/workflows/playwright.yml (created by npm init playwright@latest)
name: Playwright Tests
on:
  push:
    branches: [ main, master ]
  pull_request:
    branches: [ main, master ]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-node@v6
        with:
          node-version: lts/*
      - name: Install dependencies
        run: npm ci
      - name: Install Playwright Browsers
        run: npx playwright install --with-deps
      - name: Run Playwright tests
        run: npx playwright test
      - uses: actions/upload-artifact@v4
        if: ${{ !cancelled() }}                # upload even when tests fail
        with:
          name: playwright-report
          path: playwright-report/
          retention-days: 30
```

Step	Why
`npm ci`	Clean, exact install from the lock file (repeatable)
`install --with-deps`	Browsers **and** Linux libraries they need
`if: ${{ !cancelled() }}`	Report is uploaded **even if tests failed** (that's when you need it)
View results	Download the **playwright-report** artifact (zip) → extract → `npx playwright show-report <folder>` → click the trace icon
Timeouts	Prefer `globalTimeout` in config over a job timeout, so the reporter still writes a report

Scaling with sharding (split tests across machines):

```yaml
strategy:
  fail-fast: false
  matrix:
    shardIndex: [1, 2, 3, 4]
    shardTotal: [4]
steps:
  - run: npx playwright test --shard=${{ matrix.shardIndex }}/${{ matrix.shardTotal }}
# config: reporter: process.env.CI ? 'blob' : 'html'
# final job: npx playwright merge-reports --reporter html ./all-blob-reports
```

Other CI	Key idea
Docker	Use the official image `mcr.microsoft.com/playwright:v<version>-noble` (browsers + deps pre-installed; match your Playwright version)
Jenkins / Azure DevOps / GitLab	Same 4 steps: `npm ci` → `npx playwright install --with-deps` → `npx playwright test` → publish `playwright-report/` (and JUnit via `--reporter=junit`)
Secrets	Test credentials/URLs as CI secrets → `process.env.TEST_PWD`; never in code

> 🧠 CI in 4 words: **"ci, install, test, upload"**. Report needs a server: **show-report**, not double-click.

8 · Real-Time Usage & Interview Questions

Real project practice	How
Page Object Model	`class LoginPage { constructor(private page: Page) {} async login(id, pwd) {...} }`: locators + actions per page
Custom fixtures	`const test = base.extend<{ loginPage: LoginPage }>({ loginPage: async ({ page }, use) => { await use(new LoginPage(page)); } })`: inject page objects, test data
Login once (auth state)	A **setup project** logs in and saves `storageState` (cookies/localStorage) to `.auth/user.json`; other projects reuse it: faster, fewer flaky logins
Environments	`baseURL` from `process.env.BASE_URL`; `.env` files for QA/UAT/staging
API testing	`request` fixture: `const res = await request.post('/api/transfer', { data })`; `await expect(res).toBeOK()`; also for test data setup
Network mocking	`await page.route('**/api/rates', r => r.fulfill({ json: { fd: 7.1 } }))`: test edge cases without the backend
Tags & suites	`{ tag: ['@smoke', '@payments'] }` → `--grep @smoke` for PR runs; full regression nightly
Visual testing	`await expect(page).toHaveScreenshot()`; update with `--update-snapshots`

Interview question	Short answer
Why Playwright over Selenium?	Auto-waiting + web-first assertions (less flakiness), one API for Chromium/Firefox/WebKit, built-in runner, parallelism, trace viewer, codegen, API testing, network mocking, browser contexts
What is a BrowserContext?	An isolated incognito-like session inside one browser; each test gets one, so tests don't share state and run in parallel cheaply
How does Playwright reduce flakiness?	Actionability checks before actions, retrying assertions, strict locators, isolation, retries with trace on first retry
Locator strategy?	User-facing first (role, label, text), test-id when needed, CSS/XPath last; avoid nth/long paths
Hooks vs fixtures?	Hooks run code around tests; fixtures provide **reusable, on-demand, scoped** setup/teardown and are only created when a test asks for them
How do you debug a CI failure?	Download the report artifact → show-report → open the trace → check the action, DOM snapshot, network and console at the failing step
How do you speed up a big suite?	`fullyParallel`, workers, sharding across machines with blob reports merged, auth state reuse, API for data setup, tags for smoke vs regression
workers vs shards?	Workers = parallel processes on **one machine**; shards = split the suite across **several machines**

> *"In our banking projects I use Playwright with TypeScript: user-facing locators (getByRole, getByLabel, test IDs agreed with developers), web-first assertions that auto-wait, the Page Object Model with custom fixtures, and a setup project that logs in once and reuses storageState. Tests are tagged so pull requests run @smoke and nightly runs the full regression across Chromium, Firefox and WebKit. On GitHub Actions we run npm ci, install browsers with dependencies, shard the suite across machines with blob reports merged into one HTML report, retry twice on CI with trace on first retry, and always upload the report, so any failure can be debugged in the trace viewer with DOM snapshots, network and console."*

9 · Playwright Test Agents: Planner → Generator → Healer

Agents: three **AI agents** that ship with Playwright and run inside your AI coding tool (VS Code Copilot, Claude Code, Codex, OpenCode). They use Playwright's own browser tools (MCP) to **explore the real app**, **write the test plan**, **write the tests** and **fix failing tests**. Run them one by one or chained as a loop.

```
1. PLANNER    IN : prompt ("plan fund transfer tests") + tests/seed.spec.ts (+ PRD)
              DOES: runs the seed, explores the app in a real browser
              OUT: specs/fund-transfer.md  (Markdown test plan)           --+
                                                                           |
2. GENERATOR  IN : specs/fund-transfer.md                                <-+
              DOES: performs each step live, verifies locators + assertions
              OUT: tests/transfer/*.spec.ts                               --+
                                                                           |  run tests: fail?
3. HEALER     IN : name of the failing test                              <-+
              DOES: replays, inspects the UI, patches locator / wait / data, re-runs
              OUT: passing test  OR  test skipped (feature really broken = defect)
```

Agent	Input	What it does	Output	QA lead view
Planner	Your request ("plan tests for fund transfer"), the **seed test** `tests/seed.spec.ts`, optional **PRD**	Runs the seed (so global setup, fixtures, hooks, login all happen), then **explores the app** and designs scenarios	**Markdown plan** in `specs/`, e.g. `specs/fund-transfer.md`	Like a test analyst writing test cases. **Review the plan** before generating
Generator	The Markdown plan (name it in the prompt)	Performs each step in a real browser, **checks locators and assertions live**, writes code	Test files in `tests/`, one per scenario where feasible	Like an automation engineer. Code review it like any PR
Healer	The **name of a failing test**	Replays the failing steps, inspects the current UI, proposes a fix (locator, wait, data), **re-runs until it passes** or guardrails stop it	A **passing test**, or the test **skipped** if it decides the feature is really broken	Like a maintenance engineer. A "skip" means: raise a **defect**, don't hide it

Installation & files

```
npx playwright init-agents --loop=vscode      # VS Code (needs VS Code 1.105+)
npx playwright init-agents --loop=claude      # Claude Code
npx playwright init-agents --loop=codex       # Codex
npx playwright init-agents --loop=opencode    # OpenCode
# Re-run after every Playwright upgrade so the agents get the new tools/instructions (don't hand-edit them)
```

```
repo/
  .github/                          agent definitions (instructions + Playwright MCP tools)
  specs/                            human-readable test plans (Planner output, Generator input)
    fund-transfer.md
  tests/
    seed.spec.ts                    SEED test: shared starting point for all agents
    transfer/transfer-to-saved-payee.spec.ts     generated tests (Generator output, Healer input)
  playwright.config.ts
```

The files passed between agents (banking example, illustrative)

```
// tests/seed.spec.ts : gives the agents a ready page (logged in via your fixtures/setup project)
import { test, expect } from './fixtures';
test('seed', async ({ page }) => {
  await page.goto('/dashboard');            // planner starts exploring from here
});
```

```
# specs/fund-transfer.md   (PLANNER output -> GENERATOR input)
# NetBanking - Fund Transfer Test Plan
## Application Overview
Logged-in customer can transfer money to saved payees, with OTP and daily limit.
## Test Scenarios
### 1. Transfer to a saved payee
**Seed:** `tests/seed.spec.ts`
#### Steps
1. Click "Transfer" in the menu
2. Select payee "Ravi Kumar"
3. Type "500" in Amount and click "Continue"
4. Enter OTP "123456" and click "Confirm"
#### Expected Results
- "Transfer successful" message is visible
- Available balance is reduced by 500
### 2. Amount above daily limit is rejected
...
```

```
// tests/transfer/transfer-to-saved-payee.spec.ts   (GENERATOR output -> HEALER input if it fails)
// spec: specs/fund-transfer.md
// seed: tests/seed.spec.ts
import { test, expect } from '../fixtures';

test('Transfer to a saved payee', async ({ page }) => {
  // 1. Click "Transfer" in the menu
  await page.getByRole('link', { name: 'Transfer' }).click();
  // 2. Select payee "Ravi Kumar"
  await page.getByLabel('Payee').selectOption('Ravi Kumar');
  // 3. Type "500" in Amount and click "Continue"
  await page.getByLabel('Amount').fill('500');
  await page.getByRole('button', { name: 'Continue' }).click();
  // 4. Enter OTP and confirm
  await page.getByLabel('OTP').fill('123456');
  await page.getByRole('button', { name: 'Confirm' }).click();
  await expect(page.getByText('Transfer successful')).toBeVisible();
});
```

Healer example: developers rename the button "Continue" → "Proceed". The test fails; you ask the healer to fix "*Transfer to a saved payee*". It replays, sees the new button, patches the locator to `getByRole('button', { name: 'Proceed' })`, re-runs, and it passes. If instead the transfer page throws a server error, it **skips** the test: that's a real defect to log.

Where & how to use in a project	Guardrails a QA lead adds
• New feature: Planner from the PRD/user story → review plan → Generator • Expand coverage for legacy screens quickly • Locator breakage after UI changes: Healer • Keep `specs/*.md` as living test documentation	• Human review of plans and every generated/healed diff (PR) • Run on **test environments with test data** only, never production • Keep your POM/fixtures as the standard; seed imports them • Don't let "healed by skipping" hide real bugs • Re-run `init-agents` after upgrades

🧠 **"Plan it, Generate it, Heal it"** = analyst → automation engineer → maintenance engineer. Files: **seed.spec.ts** → **specs/*.md** → **tests/*.spec.ts**.

> *"Playwright Test Agents are three AI agents: the Planner explores the app from a seed test, optionally with a PRD, and writes a Markdown test plan in specs; the Generator turns that plan into Playwright tests, validating locators and assertions in a live browser; and the Healer takes a failing test, replays it, and patches locators, waits or data until it passes, or skips it if the feature is genuinely broken. I set them up with npx playwright init-agents for our AI tool, make the seed test use our fixtures and login setup, and keep humans in the loop: plans and generated or healed code are reviewed in PRs, and any test the healer skips becomes a defect."*

9B · Instructing the Agents: Context Files + Prompts

The agents work best with **two layers of instructions**: a **context file** (the team's permanent rules: the "system prompt") and a **task prompt** (this scenario: steps, data, expected results). Your reference files (*playwright-context.md*, *playwright-context-pom.md*, *testcases_prompts.md*) follow exactly this idea.

```
LAYER 1  CONTEXT FILE (once per repo)   role, MCP-first rule, phases, locator rules,
         docs/ai/playwright-context-pom.md   POM/fixtures, do-not rules, done criteria
LAYER 2  TASK PROMPT (per scenario)      preconditions, numbered steps, test data,
                                         EXACT expected results, tags, file names
AGENT    Planner / Generator / Healer    built-in loop (init-agents) + Playwright MCP tools
```

Is this better than the built-in agents alone? Verdict

	Built-in agents only (Section 9)	Built-in agents + your context file + structured prompts
Locators verified live	Yes (generator checks in a browser)	Yes, and made a **hard rule** ("never hallucinate selectors")
Follows OUR framework	Generic code; may use plain `page` calls	**Yes**: POM folders, readonly locators, no asserts in pages, our naming
Code-quality rules	Playwright defaults	Explicit: role locators, no XPath/nth-child, no hard waits, `test.describe`, atomic tests
Done criteria	Test passes or is skipped	Success checklist + **MCP summary** (pages, elements, locator strategy) for review
Consistency across people	Depends on each person's prompt	**Same rules for everyone** (file in the repo, reviewed like code)
Gaps in the reference files	–	No Planner/Healer rules, credentials in prompts, "iterate until zero failures" can tempt the AI to weaken assertions, no seed/fixtures/tags/test-data rules

> **Verdict:** keep the built-in Planner → Generator → Healer loop and **add** a team context file plus structured prompts. Don't edit the agent definition files (`init-agents` regenerates them); put team rules in your own file and reference it in every prompt (or in your tool's always-on instructions file, e.g. `.github/copilot-instructions.md` for VS Code Copilot, `CLAUDE.md` for Claude Code).

Improved team context file (based on your files + the gaps)

```
# docs/ai/playwright-context-pom.md : TEAM RULES FOR PLAYWRIGHT AGENTS
## ROLE
Expert Playwright (TypeScript) test engineer for this repo. Produce reliable, maintainable tests
that follow OUR framework, verified in a real browser through Playwright MCP.

## MCP-FIRST (STRICT)
- Explore the real app with MCP tools before writing code. Never invent selectors or DOM structure.
- Start every session from tests/seed.spec.ts (it logs in via our fixtures / storageState).

## FRAMEWORK RULES
- Specs in tests/<feature>/*.spec.ts; page objects in pages/<Name>Page.ts; shared widgets in components/.
- Import { test, expect } from '../fixtures' (never from '@playwright/test' in specs).
- Page objects: readonly locators + business methods; NO assertions inside page objects.
- Locators: getByRole > getByLabel > getByPlaceholder > getByText > getByTestId('data-qa').
  No XPath, nth-child, long CSS chains, or .first()/.nth() unless explained in a comment.
- Assertions: web-first only (await expect(locator)...). Assert the EXACT expected result given.
- Waits: auto-waiting, toHaveURL, waitForResponse. Never waitForTimeout, never { force: true }.
- Data: unique per test (timestamp/run ID) or from data fixtures. Credentials ONLY via process.env.
- Tag every test (@smoke / @regression + feature tag). One behaviour per test; tests independent.

## WORKFLOW
1 Explore (MCP) -> 2 Design page objects -> 3 Write spec -> 4 Run it -> 5 Fix -> 6 Report.

## FIXING RULES (GENERATOR + HEALER)
- Fix locators, waits and test data. NEVER change an expected result or delete an assertion
  just to make a test pass.
- If the app behaves differently from the expected result, stop: mark the test
  test.fixme('BUG: <what differs>') and report it as a possible defect.
- Max 3 fix attempts per test, then report what blocks you.

## OUTPUT
- Files created/changed, then an MCP SUMMARY: pages visited, elements inspected,
  locator chosen for each element and why, assumptions (should be none), open questions.

## DO NOT
- Skip MCP exploration · hard-code secrets · use unstable selectors · add hard waits ·
  put test logic in page objects · leave partial code or console.log debugging.

## DONE WHEN
- Every interaction verified via MCP · spec passes 3 runs in a row (--repeat-each=3) ·
  lint passes · page objects reused, not duplicated · summary provided.
```

Prompt templates per agent

```
# 1) PLANNER prompt
Follow docs/ai/playwright-context-pom.md.
Using seed tests/seed.spec.ts, explore the "Admin > User Management" area of the app.
Create a test plan in specs/admin-user-management.md covering:
- happy paths, validations (empty/invalid fields), permissions (non-admin user), edge cases.
For each scenario give: title, preconditions, numbered steps, EXACT expected results, tags.
Do not write test code. List any behaviour you could not confirm as "Open questions".
```

```
# 2) GENERATOR prompt
Follow docs/ai/playwright-context-pom.md.
Generate tests for scenarios 1-4 of specs/admin-user-management.md.
- Reuse existing page objects in pages/ (extend them; don't duplicate).
- Save specs under tests/admin/, tag @regression @admin.
- Verify every locator live with MCP, run each test, and finish with the MCP summary.
```

```
# 3) HEALER prompt
Follow docs/ai/playwright-context-pom.md (FIXING RULES section).
Fix the failing test "Admin can search system user by username" in tests/admin/search-user.spec.ts.
- Replay it, compare with the current UI, fix locators/waits/data in the PAGE OBJECT if the
  element changed (not in the spec).
- Do NOT change expected results. If the app is wrong, mark test.fixme with the reason.
- Re-run 3 times; report root cause, the change made, and whether it is a test or app issue.
```

Scenario prompts: your examples, improved

In the reference prompts	Better
Username/password typed in the prompt	"Log in as the admin user from the seed/fixture" or `process.env.ADMIN_USER` (fine for the public OrangeHRM demo, never for a bank)
"Verify search results are displayed"	Exact expectation: "results table shows a row whose Name column contains 'John123'"
"Verify the post appears in the feed"	"first post in Latest Posts contains the exact text; use a unique message with a timestamp"
Fixed data "John123 Doe456"	Unique names per run (`John_<timestamp>`) + clean-up step, so reruns and parallel runs don't clash
"Ensure the browser is closed" / "log pass/fail"	Not needed: the runner closes browsers and the reporter shows each data row (one test per row)
Prompt 7: three cases in one loop	"Create one test per data row with the username in the title" (Section 18 pattern)
No file names / tags	"Save as tests/pim/add-employee.spec.ts, tag @smoke @pim, use PimPage and AddEmployeePage"

```
# Improved version of "Prompt 5: Add New Employee"
Follow docs/ai/playwright-context-pom.md.
Scenario: Add a new employee (OrangeHRM demo, baseURL from config)
Preconditions: logged in as admin via the seed test.
Steps:
 1. Open "PIM" from the left menu, then "Add Employee".
 2. First name "Auto_<timestamp>", last name "Tester".
 3. Click "Save".
Expected:
 - URL contains "/pim/viewPersonalDetails".
 - Heading shows "Auto_<timestamp> Tester".
Clean-up: delete the employee via PIM > Employee List search + Delete (or API) in afterEach.
Output: pages/PimPage.ts, pages/AddEmployeePage.ts (reuse if they exist),
        tests/pim/add-employee.spec.ts tagged @smoke @pim, then the MCP summary.
```

🧠 **"Rules once, prompt per scenario."** Context file = HOW we write tests · Task prompt = WHAT to test (steps + exact expected) · Healer fixes the test, **never the expectation**.

> *"With Playwright's agents I use two layers of instructions. A context file in the repo acts as the system prompt for all three agents: MCP-first so every locator is verified in a real browser, our page-object and fixture structure, the locator priority, no hard waits, credentials only from environment variables, and fixing rules that stop the AI from weakening assertions; if the app behaves differently, it marks the test fixme and reports a possible defect. Then each task gets a structured prompt: preconditions, numbered steps, exact expected results, tags and file names. The planner writes the Markdown plan, the generator produces page objects and specs with an MCP summary for review, and the healer fixes locators in page objects rather than in specs. Every output goes through a normal pull-request review."*

10 · Annotations: Control Which Tests Run and How

Annotations: markers on tests or groups that change how they run (focus, skip, expect failure, give more time) or add metadata (tags, issue links) for filtering and reporting.

Annotation	Meaning	Real-time use
`test.only(...)` / `test.describe.only`	Run **only** these	Local debugging. Blocked on CI by `forbidOnly`
`test.skip(...)` / `test.describe.skip`	Don't run	Feature not deployed in this environment
`test.skip(condition, 'reason')`	Skip **conditionally**	Not supported on Safari / mobile / UAT
`test.fixme(...)`	Known broken: don't run, mark "to fix"	Open defect; keep the test visible in reports
`test.fail()`	**Expected** to fail: passes if it fails, alerts if it starts passing	Known bug; tells you when the fix lands
`test.slow()`	Triples the timeout	Statement download, large reports
`{ tag: '@smoke' }` or `@smoke` in title	Label for filtering	PR = @smoke, nightly = @regression
`{ annotation: { type, description } }`	Custom metadata shown in the report	Link Jira ID / issue URL
`test.info().annotations.push(...)`	Add metadata **at runtime**	Record browser version, test account used
`test.describe.configure({ mode: 'serial' })`	Run a group in order; stop after the first failure	Dependent steps (create payee → transfer → delete payee)

```
import { test, expect } from '@playwright/test';

test.describe('Fund transfer', { tag: '@payments' }, () => {

  test('transfer to saved payee', {
    tag: ['@smoke', '@regression'],
    annotation: { type: 'issue', description: 'https://jira.bank.com/browse/NB-1423' },
  }, async ({ page }) => { /* ... */ });

  test('transfer via UPI', async ({ page, browserName }) => {
    test.skip(browserName === 'webkit', 'UPI widget not supported on Safari yet');
    /* ... */
  });

  test('download yearly statement PDF', async ({ page }) => {
    test.slow();                                  // 3x timeout
    /* ... */
  });

  test('IMPS limit message', async ({ page }) => {
    test.fail();                                  // known bug NB-1500: wrong limit text
    await expect(page.getByText('Daily limit ₹5,00,000')).toBeVisible();
  });

  test.fixme('scheduled transfer edit', async ({ page }) => { /* broken: NB-1511 */ });
});

test.describe('Mobile only', () => {
  test.skip(({ isMobile }) => !isMobile, 'Mobile layout only');   // group-level condition
  test('bottom navigation works', async ({ page }) => { /* ... */ });
});

test('records test account', async ({ page }) => {
  test.info().annotations.push({ type: 'account', description: 'C123456 (UAT)' });
});
```

```
npx playwright test --grep @smoke                          # only smoke
npx playwright test --grep-invert @slow                    # everything except slow
npx playwright test --grep "@smoke|@payments"              # OR
npx playwright test --grep "(?=.*@smoke)(?=.*@payments)"   # AND
npx playwright test --grep --% "@smoke^|@payments"         # OR in PowerShell
```

Tags must start with `@`. Custom annotation types starting with `_` are hidden in the HTML report. Tags can also be filtered in config: `grep` / `grepInvert` (top level or per project).

> 🧠 **"Only, Skip, Fixme, Fail, Slow"** = focus, ignore, broken, expected-broken, needs-time (**OS-FFS**). Tags filter, annotations document.

> *"I use annotations to keep the suite honest and fast: tags like @smoke and @regression drive which tests run on PRs versus nightly; test.skip with a condition handles browser or environment limits; test.fixme and test.fail track known defects with Jira links in the annotation, so the report shows the reason, and test.fail tells us when the fix lands; test.slow gives heavy tests like statement downloads more time; and forbidOnly stops anyone merging a test.only."*

11 · Command Line: More Useful Options

Only options **not already listed in Section 2**.

Option / command	What it does	Real-time use
`-c, --config playwright.uat.config.ts`	Use another config file	Separate UAT / prod-smoke configs
`-j, --workers 50%`	Workers as a number or % of CPU	Tune speed on bigger CI agents
`--forbid-only`	Fail if any `test.only` exists	CI safety (if not set in config)
`--fail-on-flaky-tests`	Fail the run if a test passed only on retry	Release gate: zero flakiness allowed
`--fully-parallel`	Parallelise tests inside files too	Speed up without editing config
`--timeout 60000`	Per-test timeout (ms); 0 = unlimited	Slow test environment
`--global-timeout 3600000`	Max time for the whole run	Stop hung CI jobs but still get a report
`--only-changed=origin/main`	Run only test files changed vs a Git ref	Fast PR checks
`--no-deps`	Skip project dependencies (setup/teardown)	Re-run tests when login state already exists
`--shuffle` [seed]	Random order (seed to reproduce)	Find tests that depend on each other
`--test-list failed.txt` / `--test-list-invert`	Run / skip tests listed in a file	Re-run a quarantine list
`--add-reporter junit`	Add a reporter on top of the config ones	Extra JUnit XML for Jenkins/Azure
`--output results/`	Folder for traces/screenshots/videos	Custom artifact path on CI
`--ignore-snapshots`	Skip screenshot assertions	Run functional checks on a different OS
`-u changed` (all, changed, missing, none)	Update snapshots by mode	Accept intended UI changes only
`--pass-with-no-tests`	Exit 0 when nothing matches	Tag-filtered pipelines that may be empty
`--quiet` · `--tsconfig <path>`	Less output · use one tsconfig for all files	Clean CI logs · monorepos
`--ui-port=8080 --ui-host=0.0.0.0`	UI mode served on a port/host	UI mode from a container/VM
Other commands		
`npx playwright install --list` · `--force` · `--only-shell`	List installed · reinstall · only headless shell	Fix corrupted browsers; slimmer CI images
`npx playwright install-deps` · `uninstall`	Only OS dependencies · remove browsers	Prepare Linux agents · free disk space
`codegen -o tests/login.spec.ts --target javascript -b firefox`	Save to file · output language · browser	Record directly into a file
`codegen --test-id-attribute=data-qa`	Use your team's test-id attribute	Recorder prefers `data-qa`
`show-report --port 9400` · `show-trace --port 9401`	Custom port (default report port 9323)	Port already in use
`npx playwright clear-cache`	Clear Playwright caches	Strange compile/caching issues
`npx playwright init-agents --loop=vscode`	Install AI test agents (Section 9)	Planner/Generator/Healer

🧠 CI favourites: **--forbid-only · --fail-on-flaky-tests · --only-changed · --global-timeout · --add-reporter junit**.

12 · Configuration: Useful Options, Global Setup & Teardown

```
import { defineConfig, devices } from '@playwright/test';
import dotenv from 'dotenv';
dotenv.config({ path: `.env.${process.env.TEST_ENV ?? 'qa'}` });     // .env.qa / .env.uat

export default defineConfig({
  testDir: './tests',
  testMatch: '**/*.spec.ts',                // which files are tests
  testIgnore: '**/drafts/**',               // which to skip
  timeout: 60_000,                          // per test (default 30 s)
  globalTimeout: 60 * 60 * 1000,            // whole run (CI safety)
  expect: { timeout: 10_000,                // web-first assertion wait (default 5 s)
            toHaveScreenshot: { maxDiffPixels: 10 } },
  fullyParallel: true,
  forbidOnly: !!process.env.CI,
  retries: process.env.CI ? 2 : 0,
  workers: process.env.CI ? '50%' : undefined,
  maxFailures: process.env.CI ? 20 : 0,     // stop a broken build early
  outputDir: 'test-results',                // traces, screenshots, videos
  reporter: [['html', { open: 'never' }], ['junit', { outputFile: 'results/junit.xml' }], ['list']],
  use: { baseURL: process.env.BASE_URL, trace: 'on-first-retry',
         screenshot: 'only-on-failure', video: 'retain-on-failure' },
  webServer: {                              // start the app before tests (local/dev)
    command: 'npm run start', url: 'http://localhost:3000',
    reuseExistingServer: !process.env.CI,
  },
  projects: [ /* setup, browsers, teardown: see below */ ],
});
```

Option	Real-time use
`timeout` / `expect.timeout` / `globalTimeout`	Per test (30 s) / per assertion (5 s) / whole run. Raise for slow UAT; keep `globalTimeout` on CI
`retries` + `trace: 'on-first-retry'`	Absorb rare infra flakiness on CI and capture evidence
`reporter` (array)	HTML for people, JUnit for Jenkins/Azure, list for console; `blob` when sharding
`webServer`	Starts the front-end locally before tests; reuses a running one
`testMatch` / `testIgnore` / `grep`	Control which files/tests are picked up
`dotenv` + `TEST_ENV`	One suite, many environments (QA/UAT): `TEST_ENV=uat npx playwright test`

Global setup & teardown: Option 1, project dependencies (recommended)

```
// playwright.config.ts
projects: [
  { name: 'setup', testMatch: /global\.setup\.ts/, teardown: 'cleanup' },
  { name: 'cleanup', testMatch: /global\.teardown\.ts/ },
  { name: 'chromium', use: { ...devices['Desktop Chrome'], storageState: '.auth/user.json' },
    dependencies: ['setup'] },
  { name: 'firefox',  use: { ...devices['Desktop Firefox'], storageState: '.auth/user.json' },
    dependencies: ['setup'] },
],
```

```
// tests/global.setup.ts : runs ONCE before all dependent projects (it's a normal test)
import { test as setup, expect } from '@playwright/test';

setup('create test data and log in', async ({ page, request }) => {
  const res = await request.post('/api/test-data/payees', { data: { name: 'Ravi Kumar' } });
  await expect(res).toBeOK();                                    // seed data via API
  await page.goto('/login');
  await page.getByLabel('Customer ID').fill(process.env.TEST_USER!);
  await page.getByLabel('Password').fill(process.env.TEST_PWD!);
  await page.getByRole('button', { name: 'Log in' }).click();
  await expect(page).toHaveURL(/dashboard/);
  await page.context().storageState({ path: '.auth/user.json' });  // save login for all tests
});

// tests/global.teardown.ts : runs ONCE after all dependent projects finish
import { test as teardown } from '@playwright/test';
teardown('delete test data', async ({ request }) => {
  await request.delete('/api/test-data/payees/Ravi%20Kumar');
});

// tests/dashboard.spec.ts : starts ALREADY logged in (storageState)
test('balance card shows', async ({ page }) => {
  await page.goto('/dashboard');
  await expect(page.getByTestId('balance-amount')).toBeVisible();
});
```

Option 2: `globalSetup` / `globalTeardown` functions

```
// playwright.config.ts
globalSetup: require.resolve('./global-setup'),
globalTeardown: require.resolve('./global-teardown'),
use: { baseURL: 'https://uat.bank.com', storageState: 'state.json' },

// global-setup.ts : plain function, no fixtures, launch the browser yourself
import { chromium, type FullConfig } from '@playwright/test';
export default async function globalSetup(config: FullConfig) {
  const { baseURL, storageState } = config.projects[0].use;
  const browser = await chromium.launch();
  const page = await browser.newPage();
  await page.goto(baseURL! + '/login');
  await page.getByLabel('Customer ID').fill(process.env.TEST_USER!);
  await page.getByLabel('Password').fill(process.env.TEST_PWD!);
  await page.getByRole('button', { name: 'Log in' }).click();
  await page.context().storageState({ path: storageState as string });
  process.env.RUN_ID = Date.now().toString();      // share simple data with tests via env
  await browser.close();
}
```

	Project dependencies	`globalSetup` function
Shown in HTML report / trace	Yes / yes	No / no
Fixtures (`page`, `request`), config `use` options	Yes, applied automatically	No: launch browsers manually
Retries, parallelism	Yes	Not applicable
Skip it	`--no-deps`	Remove from config

> 🧠 **"Setup project logs in once, saves storageState; tests reuse it; teardown cleans up."** Prefer **projects over globalSetup**: you get report, trace and fixtures.

13 · Configuration "use": Options & Scope (Global → Project → File → Describe)

`use` holds options for the **browser, context and page** that every test gets (base URL, login state, device, locale, recording). Set it at different levels; **the most specific level wins**.

```
GLOBAL   playwright.config.ts  use: {...}            applies to every test
  PROJECT  projects[].use: {...}                     overrides global for that browser/device/env
    FILE     test.use({...}) at top of a spec file   overrides for all tests in that file
      DESCRIBE test.use({...}) inside test.describe  overrides for that group only   <- WINS
```

Group	Options	Real-time use
Basic	`baseURL`, `storageState`	Env switching; start logged in
Emulation	`viewport`, `locale`, `timezoneId`, `colorScheme`, `geolocation` + `permissions`, `userAgent`, `isMobile`, `hasTouch`, `deviceScaleFactor`, `offline`, `javaScriptEnabled`	Arabic/UAE customers, branch locator, dark mode, offline banner, mobile web
Network	`extraHTTPHeaders`, `httpCredentials`, `ignoreHTTPSErrors`, `proxy`, `acceptDownloads`	API keys/feature flags in headers, basic-auth UAT, self-signed certs, corporate proxy, statement downloads
Recording	`screenshot` (off/on/only-on-failure), `video` & `trace` (off/on/retain-on-failure/on-first-retry/...)	Evidence for failures and audits
Browser / other	`browserName`, `channel` ('chrome', 'msedge'), `headless`, `launchOptions` (e.g. `slowMo`), `actionTimeout`, `navigationTimeout`, `testIdAttribute`, `bypassCSP`	Branded Edge/Chrome, slow-motion demos, custom `data-qa`

```
// 1) GLOBAL: playwright.config.ts
use: {
  baseURL: process.env.BASE_URL ?? 'https://qa.bank.com',
  storageState: '.auth/user.json',             // logged in by the setup project
  testIdAttribute: 'data-qa',                   // getByTestId('balance') -> [data-qa="balance"]
  locale: 'en-GB', timezoneId: 'Asia/Dubai',
  actionTimeout: 15_000, navigationTimeout: 30_000,
  extraHTTPHeaders: { 'x-test-run': 'playwright' },
  ignoreHTTPSErrors: true,                      // UAT self-signed certificate
  screenshot: 'only-on-failure', video: 'retain-on-failure', trace: 'on-first-retry',
},

// 2) PROJECT: per browser / device / environment
projects: [
  { name: 'edge',       use: { ...devices['Desktop Edge'], channel: 'msedge' } },
  { name: 'mobile',     use: { ...devices['Pixel 7'] } },                // isMobile, hasTouch, viewport
  { name: 'uat-basic',  use: {
      baseURL: 'https://uat.bank.com',
      httpCredentials: { username: process.env.UAT_USER!, password: process.env.UAT_PWD! },
  } },
],
```

```
// 3) FILE: tests/arabic.spec.ts : whole file in Arabic, UAE time
import { test, expect } from '@playwright/test';
test.use({ locale: 'ar-AE', timezoneId: 'Asia/Dubai' });

test('dashboard switches to right-to-left layout', async ({ page }) => {
  await page.goto('/dashboard');
  await expect(page.locator('html')).toHaveAttribute('dir', 'rtl');   // Arabic UI is RTL
});

// 4) DESCRIBE: groups with their own options
test.describe('Guest user (not logged in)', () => {
  test.use({ storageState: { cookies: [], origins: [] } });         // drop the saved login
  test('protected page redirects to login', async ({ page }) => {
    await page.goto('/dashboard');
    await expect(page).toHaveURL(/login/);
  });
});

test.describe('Branch locator', () => {
  test.use({ geolocation: { latitude: 25.2048, longitude: 55.2708 }, permissions: ['geolocation'] });
  test('nearest branch is Dubai', async ({ page }) => {
    await page.goto('/branches');
    await page.getByRole('button', { name: 'Use my location' }).click();
    await expect(page.getByTestId('nearest-branch')).toContainText('Dubai');
  });
});

test.describe('Offline and dark mode', () => {
  test.use({ colorScheme: 'dark', offline: true });
  test('offline banner appears', async ({ page }) => { /* ... */ });
});

test.describe('Wide screen statements', () => {
  test.use({ viewport: { width: 1920, height: 1080 }, acceptDownloads: true });
  test('download PDF statement', async ({ page }) => {
    const download = page.waitForEvent('download');
    await page.getByRole('button', { name: 'Download PDF' }).click();
    expect((await download).suggestedFilename()).toContain('.pdf');
  });
});
```

Rule	Detail
Test-runner options vs `use`	`timeout`, `retries`, `workers`, `reporter` are **top level**; browser/context options go in `use`
`test.use()` placement	Top of a file or inside `describe`, **not inside a test**
Devices in describe	Spreading `devices['iPhone 13']` in a describe fails (it changes the browser type): put devices in a **project**, or set only `viewport`/`isMobile`/`hasTouch`
Reset to config value	`test.use({ baseURL: undefined })` inside a describe

🧠 **"Global sets the default, Project sets the platform, File sets the theme, Describe sets the case"**: the closest `use` wins.

> *"Our config's global use sets baseURL from the environment, storageState from the setup project, our data-qa test-id attribute, timeouts, and recording on failure. Projects override that per platform: Chrome, Edge via channel, a Pixel device for mobile web, and a UAT project with HTTP credentials. Inside spec files, test.use handles special cases: Arabic locale and Dubai timezone for UAE customers, an empty storageState for guest tests, geolocation with permissions for the branch locator, and offline or dark mode. The closest level wins, so tests stay short and the setup lives in config."*

14 · Emulation: Mobiles & Tablets

Emulation: Playwright makes a desktop browser **behave like a phone or tablet**: screen size (`viewport`), pixel density (`deviceScaleFactor`), mobile user agent, mobile layout (`isMobile`), touch (`hasTouch`) and the matching engine. It is **not** a real device: it tests your **responsive web app**, not native apps or device hardware.

Device preset	Engine used	What it sets
`devices['iPhone 14']`, `['iPhone 13 landscape']`	**WebKit** (Safari engine)	`viewport`, `userAgent`, `deviceScaleFactor`, `isMobile: true`, `hasTouch: true`, `defaultBrowserType`. Full list: `playwright.devices` (deviceDescriptorsSource.json)
`devices['Pixel 7']`, `['Galaxy S9+']`	**Chromium** (Chrome engine)	
`devices['iPad Pro 11']`, `['iPad (gen 7) landscape']`	**WebKit**	
`devices['Galaxy Tab S4']`	**Chromium**	

Firefox doesn't support `isMobile`, so mobile presets run on WebKit or Chromium.

Where to change: `playwright.config.ts` projects (main place)

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  use: { baseURL: process.env.BASE_URL },
  projects: [
    { name: 'Desktop Chrome', use: { ...devices['Desktop Chrome'] } },

    // ---------- MOBILES ----------
    { name: 'Mobile Chrome',  use: { ...devices['Pixel 7'] },   grep: /@mobile|@smoke/ },
    { name: 'Mobile Safari',  use: { ...devices['iPhone 14'] }, grep: /@mobile|@smoke/ },

    // ---------- TABLETS ----------
    { name: 'Tablet iPad',    use: { ...devices['iPad Pro 11 landscape'] }, grep: /@tablet|@smoke/ },
    { name: 'Tablet Android', use: { ...devices['Galaxy Tab S4'] },          grep: /@tablet|@smoke/ },

    // custom device: preset + overrides (e.g. small phone, Arabic, Dubai)
    { name: 'Small phone AR', use: { ...devices['iPhone SE'], locale: 'ar-AE', timezoneId: 'Asia/Dubai' } },
  ],
});
```

Where else	When
`test.use({ viewport: { width: 390, height: 844 } })`	Only screen size for one file/describe (not a whole device preset: that must be a project)
`await page.setViewportSize({ width: 768, height: 1024 })`	Resize **during** a test (responsive breakpoints)
`({ isMobile, browserName, viewport })` fixtures	Branch or skip inside a test: `test.skip(!isMobile, 'mobile only')`

Run on devices

```
npx playwright test --project="Mobile Safari"           # one phone
npx playwright test --project="Mobile*"                  # all mobiles (wildcard)
npx playwright test --project="Tablet*" --grep @tablet   # tablets, tablet tests only
npx playwright codegen --device="iPhone 14" https://qa.bank.com   # record on a phone
```

Example tests

```
import { test, expect } from '@playwright/test';

// 1) Mobile: hamburger menu instead of top navigation
test('mobile menu opens Transfers', { tag: '@mobile' }, async ({ page, isMobile }) => {
  test.skip(!isMobile, 'Mobile layout only');
  await page.goto('/dashboard');
  await expect(page.getByRole('navigation', { name: 'Main' })).toBeHidden();   // desktop nav hidden
  await page.getByRole('button', { name: 'Menu' }).tap();                       // tap() needs hasTouch
  await page.getByRole('link', { name: 'Transfers' }).tap();
  await expect(page).toHaveURL(/transfers/);
});

// 2) Tablet: statement table shows all columns in landscape, fewer after rotating to portrait
test('statement columns adapt to orientation', { tag: '@tablet' }, async ({ page }) => {
  await page.goto('/statements');
  await expect(page.getByRole('columnheader', { name: 'Reference' })).toBeVisible();   // landscape
  await page.setViewportSize({ width: 834, height: 1194 });                            // portrait
  await expect(page.getByRole('columnheader', { name: 'Reference' })).toBeHidden();
});
```

Real-time practice	Detail
Which devices?	Pick from **analytics** (top 2–3 phones + 1 tablet per OS); run @smoke on all, full regression on desktop + 1 phone
Tag tests	`@mobile`, `@tablet` + project `grep` so desktop-only tests don't run on phones
Touch	Use `tap()` on mobile projects; `click()` also works
Visual checks	`toHaveScreenshot()` per project: baselines are stored per project/device
Limits	No real iOS Safari/Chrome, no native app, hardware or real network: use a real-device cloud for those

🧠 **"Device = project, size = test.use, rotate = setViewportSize."** iPhone/iPad → WebKit · Pixel/Galaxy → Chromium · Firefox has no isMobile.

> *"Mobile and tablet coverage is done with device emulation in playwright.config.ts: separate projects spread Playwright's device presets, such as Pixel 7 and iPhone 14 for phones and iPad Pro 11 landscape and Galaxy Tab S4 for tablets, which sets viewport, user agent, scale factor, isMobile and hasTouch on the right engine. Tests are tagged @mobile or @tablet and filtered with project grep, we run them with --project="Mobile*", and inside tests we use the isMobile fixture, tap() and setViewportSize for orientation. Emulation covers our responsive web app; real-device clouds cover what emulation can't."*

15 · Fixtures: Role-Based Users & Lead-Level Fixtures

Fixtures: reusable **setup** → **use** → **teardown** blocks that Playwright **injects by name** into tests (`{ page }` is one). Created only when a test asks for them; dependencies are set up first and torn down last.

Built-in fixture	Gives you	Fixture feature	Syntax
`page`	Isolated page for this test	Test scope (default)	`name: async ({ page }, use) => {...}`
`context`	Its browser context (cookies, storage)	Worker scope (once per worker)	`[async ({}, use, workerInfo) => {...}, { scope: 'worker' }]`
`browser`	Shared browser (new contexts)	Automatic (runs without being asked)	`[fn, { auto: true }]`
`request`	API client	Option (set from config / test.use)	`role: ['customer', { option: true }]`
`browserName`	chromium / firefox / webkit	Timeout · hide in report · combine	`{ timeout: 60_000 }` · `{ box: true }` · `mergeTests(a, b)`

1) The key fixture: role-based users (customer / maker / checker / admin)

```
// tests/auth.setup.ts : setup project logs in ONCE per role, saves storageState per role
import { test as setup, expect } from '@playwright/test';

const roles = ['customer', 'maker', 'checker', 'admin'] as const;
for (const role of roles) {
  setup(`login as ${role}`, async ({ page }) => {
    await page.goto('/login');
    await page.getByLabel('User ID').fill(process.env[`${role.toUpperCase()}_ID`]!);
    await page.getByLabel('Password').fill(process.env[`${role.toUpperCase()}_PWD`]!);
    await page.getByRole('button', { name: 'Log in' }).click();
    await expect(page.getByTestId('user-menu')).toBeVisible();
    await page.context().storageState({ path: `playwright/.auth/${role}.json` });
  });
}
// config: { name: 'setup', testMatch: /auth\.setup\.ts/ } and dependencies: ['setup'] in browser projects
// .gitignore: playwright/.auth   (state files hold session cookies)
```

```ts
// fixtures/roles.ts
import { test as base, expect, type Page } from '@playwright/test';
import { PaymentsPage } from '../pages/PaymentsPage';
import { ApprovalsPage } from '../pages/ApprovalsPage';

export type Role = 'customer' | 'maker' | 'checker' | 'admin';
export const authFile = (role: Role) => `playwright/.auth/${role}.json`;

type RoleFixtures = {
  role: Role;                       // option fixture: which user the default page is
  makerPage: PaymentsPage;          // extra pages for multi-role tests
  checkerPage: ApprovalsPage;
  adminPage: Page;
};

export const test = base.extend<RoleFixtures>({
  role: ['customer', { option: true }],                        // default role
  storageState: async ({ role }, use) => use(authFile(role)),   // default `page` logs in as `role`

  makerPage: async ({ browser, baseURL }, use) => {
    const context = await browser.newContext({ baseURL, storageState: authFile('maker') });
    await use(new PaymentsPage(await context.newPage()));       // setup ... use
    await context.close();                                      // ... teardown
  },
  checkerPage: async ({ browser, baseURL }, use) => {
    const context = await browser.newContext({ baseURL, storageState: authFile('checker') });
    await use(new ApprovalsPage(await context.newPage()));
    await context.close();
  },
  adminPage: async ({ browser, baseURL }, use) => {
    const context = await browser.newContext({ baseURL, storageState: authFile('admin') });
    await use(await context.newPage());
    await context.close();
  },
});
export { expect };
```

```ts
// tests/payments.spec.ts : how tests use the role fixtures
import { test, expect } from '../fixtures/roles';

test('customer sees own accounts', async ({ page }) => {          // default role = customer
  await page.goto('/dashboard');
  await expect(page.getByRole('heading', { name: 'My Accounts' })).toBeVisible();
});

test.describe('Admin screens', () => {
  test.use({ role: 'admin' });                                      // whole group as admin
  test('admin can open user management', async ({ page }) => {
    await page.goto('/admin/users');
    await expect(page.getByRole('table', { name: 'Users' })).toBeVisible();
  });
});

test('maker-checker: payment needs approval', async ({ makerPage, checkerPage }) => {
  const ref = await makerPage.createPayment({ payee: 'Ravi Kumar', amount: 250000 });  // maker
  await checkerPage.approve(ref);                                                       // checker
  await expect(makerPage.statusOf(ref)).toHaveText('Approved');                         // maker sees it
});
```

A role can also be set per project: `{ name: 'admin-chrome', use: { role: 'admin' } }`.

2) Other fixtures a lead should create

Fixture	Type	Why
Page objects (`loginPage`, `dashboardPage`)	Test	No `new LoginPage(page)` in every test; one place to change
Test data (`payee`, `account`)	Test	Create unique data via API before, **delete after**: independent, parallel-safe tests
Console / network guard	Automatic	Every test collects browser console errors and attaches them to the report
Per-worker account	Worker	Each parallel worker uses its own user: no clashes on balances/limits
API client with token	Worker	Get an OAuth token once per worker, reuse for API setup/assertions
Environment / feature options	Option	`env: 'qa' | 'uat'`, feature flags set per project

```
// fixtures/data.ts
import { test as base, expect } from '@playwright/test';

type DataFixtures = { payee: { id: string; name: string }; consoleGuard: void };
type WorkerFixtures = { workerUser: string };

export const test = base.extend<DataFixtures, WorkerFixtures>({
  // TEST DATA: create via API, give to test, clean up after
  payee: async ({ request }, use) => {
    const name = `Payee_${Date.now()}`;
    const res = await request.post('/api/payees', { data: { name, ifsc: 'HDFC0001234' } });
    expect(res.ok()).toBeTruthy();
    const payee = await res.json();
    await use(payee);
    await request.delete(`/api/payees/${payee.id}`);
  },

  // AUTOMATIC: runs for every test without being requested
  consoleGuard: [async ({ page }, use, testInfo) => {
    const errors: string[] = [];
    page.on('console', msg => { if (msg.type() === 'error') errors.push(msg.text()); });
    await use();
    if (errors.length) {
      await testInfo.attach('console-errors', { body: errors.join('\n'), contentType: 'text/plain' });
    }
  }, { auto: true }],

  // WORKER: one test user per parallel worker
  workerUser: [async ({}, use, workerInfo) => {
    await use(`qa_user_${workerInfo.workerIndex}`);
  }, { scope: 'worker' }],
});

// fixtures/index.ts : ONE import for all tests
import { mergeTests } from '@playwright/test';
import { test as roleTest } from './roles';
import { test as dataTest } from './data';
export const test = mergeTests(roleTest, dataTest);
export { expect } from '@playwright/test';

// usage
test('pay a fresh payee', async ({ makerPage, payee }) => {
  await makerPage.createPayment({ payee: payee.name, amount: 500 });
});
```

Code written for these notes, not executed here. `PaymentsPage`/`ApprovalsPage` are your page objects.

> 🧠 Fixture = **"setup → use() → teardown"**, injected by name, lazy. Scopes: **test** (default) · **worker** (once per process) · **auto** (always) · **option** (configurable). Roles = **storageState per role**.

> *"Our framework is built on fixtures. A setup project logs in once per role (customer, maker, checker and admin) and saves a storageState file for each. A role option fixture overrides storageState, so the default page logs in as whoever the test or project asks for, for example test.use({ role: 'admin' }). For maker-checker flows, makerPage and checkerPage fixtures open separate contexts with their own sessions in the same test. Beyond roles, I add test-data fixtures that create and delete payees through the API, an automatic fixture that attaches browser console errors to the report, worker-scoped users so parallel tests don't clash, and page-object fixtures, all merged with mergeTests into a single import."*

16 · Global Setup & Teardown: What Teams Use in Real Projects

How it works (setup projects vs `globalSetup`) is in Section 12. This page lists **what teams actually put in it**.

#	Feature	Setup does	Teardown does
1	**Environment health check**	Call `/health`, check login page loads; **fail fast** if QA/UAT is down	–
2	**Authentication per role**	Log in once per role, save `storageState` files (Section 15)	Optionally log out / revoke sessions
3	**API token**	Get an OAuth/JWT token once, save to file for API tests	Revoke the token
4	**Test data seeding**	Create customers, accounts, payees, balances via API/DB; save IDs	**Delete** everything created (by run ID)
5	**Feature flags / config**	Turn on the feature under test in the test environment (admin API)	Restore original values
6	**Reset state**	Reset daily transfer limits, clear OTP/email mock inbox, unlock test users	–
7	**Mock / stub services**	Start OTP or payment-gateway stubs (`webServer` can start several servers)	Stopped automatically
8	**Run metadata**	Generate a **run ID**, record build number/environment for reports	–
9	**Database snapshot**	Snapshot or seed a known DB state (lower environments)	Restore the snapshot

```
// playwright.config.ts : chain of setup projects + one teardown
projects: [
  { name: 'health', testMatch: /health\.setup\.ts/ },
  { name: 'setup',  testMatch: /(auth|data)\.setup\.ts/, dependencies: ['health'], teardown: 'cleanup' },
  { name: 'cleanup', testMatch: /global\.teardown\.ts/ },
  { name: 'chromium', use: { ...devices['Desktop Chrome'] }, dependencies: ['setup'] },
],
webServer: [                                        // start stubs before tests
  { command: 'npm run otp-stub', url: 'http://localhost:4010/health', reuseExistingServer: !process.env.CI },
],
```

```
// tests/health.setup.ts : 1) environment must be up
import { test as setup, expect } from '@playwright/test';
setup('environment is healthy', async ({ request }) => {
  const res = await request.get('/api/health');
  expect(res.ok(), `Environment down: ${process.env.BASE_URL}`).toBeTruthy();
});

// tests/data.setup.ts : 3) token + 4) seed data + 5) feature flag, saved to a FILE for all workers
import { test as setup, expect } from '@playwright/test';
import fs from 'fs';
setup('seed test data', async ({ request }) => {
  const runId = `RUN_${Date.now()}`;
  const tokenRes = await request.post('/oauth/token', { form: {
    client_id: process.env.CLIENT_ID!, client_secret: process.env.CLIENT_SECRET!, grant_type: 'client_credentials' }
});
  const { access_token } = await tokenRes.json();
  const headers = { Authorization: `Bearer ${access_token}` };

  const customer = await (await request.post('/api/test/customers',
    { headers, data: { runId, name: 'Auto Customer', balance: 100000 } })).json();
  await request.put('/api/admin/flags/instant-transfer', { headers, data: { enabled: true } });

  fs.mkdirSync('playwright/.data', { recursive: true });
  fs.writeFileSync('playwright/.data/run.json', JSON.stringify({ runId, token: access_token, customer }));
});

// tests/global.teardown.ts : clean up by run ID, restore flag
import { test as teardown } from '@playwright/test';
import fs from 'fs';
teardown('clean test data', async ({ request }) => {
  const { runId, token } = JSON.parse(fs.readFileSync('playwright/.data/run.json', 'utf-8'));
  const headers = { Authorization: `Bearer ${token}` };
  await request.delete(`/api/test/runs/${runId}`, { headers });         // deletes everything tagged runId
  await request.put('/api/admin/flags/instant-transfer', { headers, data: { enabled: false } });
});

// any test: read the shared data
const run = JSON.parse(fs.readFileSync('playwright/.data/run.json', 'utf-8'));
test('new customer sees opening balance', async ({ page }) => { /* use run.customer.id */ });
```

Team tips	Why
Share data through **files** (or storageState), not `process.env`	Setup projects run in worker processes; env vars set there don't reach other workers (env sharing works with the `globalSetup` function)
Tag all created data with a **run ID**	Teardown deletes exactly this run's data; parallel CI runs don't collide
Teardown must be **safe to re-run**	It runs even after failures; ignore "already deleted"
Health check first	A down environment gives 1 clear failure, not 500 red tests
`--no-deps` locally	Skip setup when login/data files already exist (faster reruns)
Keep `playwright/.auth` and `.data` in `.gitignore`	They contain sessions and tokens

> 🧠 Setup = **"Check, Login, Token, Seed, Flag"** · Teardown = **"Delete, Restore, Revoke"** · Share through **files**, tag with a **run ID**.

> *"Our global setup is a chain of setup projects. A health check fails fast if the environment is down. Then we log in once per role and save storageState, get an API token, seed test data such as customers and payees through APIs tagged with a run ID, and switch on the feature flags under test. Everything is written to files under playwright/.auth and .data so all workers can read it. A teardown project deletes the run's data by run ID, restores flags and revokes tokens, even when tests fail. Stubs like the OTP service start via webServer. This keeps tests independent, parallel-safe and the environment clean."*

17 · Parallelism: Run Faster Without Flakiness

Parallelism: Playwright runs tests in several **worker processes** at once. Each worker is a separate OS process with **its own browser**; workers can't talk to each other.

Level	Default / how	Lead decision
Files	Different files run in parallel by default	Keep files small and focused so work spreads evenly
Tests inside a file	Run in order in one worker, unless `fullyParallel: true` (config or project) or `test.describe.configure({ mode: 'parallel' })`	Turn on `fullyParallel` once tests are independent; needed for good sharding
Serial group	`test.describe.configure({ mode: 'serial' })`: in order; after a failure the rest are skipped; retried as a group	Only for truly dependent flows; prefer independent tests (docs: not recommended)
Opt out of fully parallel	`test.describe.configure({ mode: 'default' })`	One legacy file that can't be parallelised yet
Number of workers	Config `workers` or `--workers 4` / `-j 50%`; `workers: 1` = no parallelism	Local: default (half the cores). CI: match agent CPUs; limit if the test environment can't take the load
Stop early	`maxFailures: 10` / `-x`	A broken build stops after N failures instead of running 2 000 red tests
Worker identity	`testInfo.workerIndex` (unique per process), `testInfo.parallelIndex` (0 … workers-1), env `TEST_PARALLEL_INDEX`	Give each worker its own test user/data

```
// playwright.config.ts
export default defineConfig({
  fullyParallel: true,
  workers: process.env.CI ? 4 : undefined,     // CI agent has 4 vCPU; QA env handles ~4 sessions
  maxFailures: process.env.CI ? 15 : 0,
});

// fixtures/worker-user.ts : one bank user per parallel slot, so balances/limits never clash
export const test = base.extend<{}, { workerUser: { id: string; pwd: string } }>({
  workerUser: [async ({}, use, workerInfo) => {
    const i = workerInfo.parallelIndex;                       // 0..workers-1, stable after restarts
    await use({ id: process.env[`QA_USER_${i}`]!, pwd: process.env.QA_USER_PWD! });
  }, { scope: 'worker' }],
});

// a flow that genuinely must stay in order (rare)
test.describe('Standing instruction lifecycle', () => {
  test.describe.configure({ mode: 'serial' });
  test('create SI', async ({ page }) => { /* ... */ });
  test('edit SI', async ({ page }) => { /* ... */ });
  test('cancel SI', async ({ page }) => { /* ... */ });
});
```

Parallel-safe rules (lead checklist)	Why
No shared mutable data between tests	Two workers transferring from the same account = random balance failures
Unique data per test (fixtures, timestamps, run ID)	Tests can run in any order, on any worker, on any shard
Don't rely on test order across files	Order isn't guaranteed in parallel
A failure restarts the worker	The next test gets a clean browser; `beforeAll` runs again in the new worker

> 🧠 **Workers = processes on one machine; fullyParallel = tests inside files too; serial = last resort.**
> "Parallel needs independent data."

> *"Files run in parallel by default; I enable fullyParallel so tests inside files spread across workers too, which also balances shards. Locally we use the default worker count; on CI we set workers to the agent's CPUs, capped by what the QA environment can handle, and maxFailures stops a broken build early. The key is independence: each worker gets its own bank user through a worker-scoped fixture keyed on parallelIndex, and data is created per test, so order doesn't matter. Serial mode is reserved for the rare flow that truly depends on order."*

18 · Parameterized (Data-Driven) Tests

Parameterize: one test body, **many data sets**. Each data row becomes its **own test** in the report (titles must be unique).

```
// 1) Inline data: boundary values for the transfer limit
const cases = [
  { amount: '1',       expected: 'Transfer successful' },
  { amount: '200000',  expected: 'Transfer successful' },                    // = daily limit
  { amount: '200001',  expected: 'Amount exceeds your daily limit' },        // limit + 1
  { amount: '0',       expected: 'Enter a valid amount' },
  { amount: '-10',     expected: 'Enter a valid amount' },
];

test.describe('IMPS amount validation', () => {
  test.beforeEach(async ({ page }) => { await page.goto('/transfer/imps'); });  // once per case

  for (const { amount, expected } of cases) {
    test(`amount ${amount} shows "${expected}"`, async ({ page }) => {
      await page.getByLabel('Amount').fill(amount);
      await page.getByRole('button', { name: 'Continue' }).click();
      await expect(page.getByTestId('transfer-message')).toHaveText(expected);
    });
  }
});
```

```
// 2) Data from a file: JSON (or CSV with csv-parse) maintained by BAs
import fs from 'fs';
import { parse } from 'csv-parse/sync';
const rows = parse(fs.readFileSync('test-data/payees.csv'), { columns: true, skip_empty_lines: true });
for (const row of rows) {
  test(`add payee ${row.case_id}: ${row.description}`, async ({ page }) => {
    await page.getByLabel('IFSC').fill(row.ifsc);
    await page.getByLabel('Account number').fill(row.account);
    await expect(page.getByTestId('payee-result')).toHaveText(row.expected);
  });
}

// 3) Per-iteration hooks: wrap each case in its own describe
for (const currency of ['INR', 'USD', 'AED']) {
  test.describe(`FX quote in ${currency}`, () => {
    test.beforeEach(async ({ page }) => { await page.goto(`/fx?currency=${currency}`); });
    test('shows rate', async ({ page }) => { /* ... */ });
    test('shows fee', async ({ page }) => { /* ... */ });
  });
}
```

```
// 4) Parameterized PROJECTS: same tests, different option value per project
// fixtures.ts
export const test = base.extend<{ segment: 'retail' | 'premium' }>({
  segment: ['retail', { option: true }],
});
// playwright.config.ts
projects: [
  { name: 'retail',  use: { segment: 'retail' } },
  { name: 'premium', use: { segment: 'premium' } },
],
// test
test('fee waiver shown only for premium', async ({ page, segment }) => {
  await page.goto('/cards/fees');
  const waiver = page.getByText('Annual fee waived');
  segment === 'premium' ? await expect(waiver).toBeVisible() : await expect(waiver).toBeHidden();
});

// 5) Environment variables as parameters
// TEST_USER=C123 npx playwright test   ->   process.env.TEST_USER in tests/config
```

Technique	Use when
Inline array + loop	Boundary values, small fixed lists
JSON / CSV file	Large data sets owned by BAs/manual QA; no code change to add cases
Describe per parameter	Each data set needs its own `beforeEach` or several tests
Option fixture + projects	Whole suite repeated per segment, role, region, language
Env variables / .env	Credentials, URLs, run-time switches

🧠 **"Loop the data, not the code; every row gets a unique title."** Small = array · big = CSV/JSON · whole suite = option + projects.

> *"For data-driven testing I loop over data to create one test per row, with the data in the title so the report shows exactly which case failed, for example boundary values for transfer limits. Larger sets, such as IFSC and payee combinations, live in CSV or JSON files maintained with the BAs. When a whole suite must run per customer segment or role, I use an option fixture and parameterized projects, and credentials and URLs come from environment variables."*

19 · Projects & Multiple Environments (Local and CI)

Project = a named group of tests + settings (browser, device, environment, role, test files). One config can hold many projects; run one with `--project`.

Local: switch environments with `.env` files

```
.env.qa       BASE_URL=https://qa.bank.com     TEST_USER=...   TEST_PWD=...
.env.uat      BASE_URL=https://uat.bank.com    TEST_USER=...   TEST_PWD=...
.env.prod     BASE_URL=https://www.bank.com    (read-only smoke user)
.gitignore    .env.*        <- never commit secrets; share a .env.example instead
```

```
// playwright.config.ts
import { defineConfig, devices } from '@playwright/test';
import dotenv from 'dotenv';

const ENV = (process.env.TEST_ENV ?? 'qa') as 'qa' | 'uat' | 'prod';
dotenv.config({ path: `.env.${ENV}` });            // local values; on CI real env vars win (dotenv doesn't
override)

const isProd = ENV === 'prod';

export default defineConfig({
  testDir: './tests',
  retries: isProd ? 0 : process.env.CI ? 2 : 0,
  reporter: process.env.CI ? [['blob'], ['github']] : [['html'], ['list']],
  tag: process.env.CI ? `@${ENV}` : undefined,        // environment label in merged reports
  use: { baseURL: process.env.BASE_URL, trace: 'on-first-retry' },
  projects: [
    { name: 'setup', testMatch: /.*\.setup\.ts/ },
    { name: 'chromium', use: { ...devices['Desktop Chrome'] }, dependencies: ['setup'],
      grep: isProd ? /@prod-smoke/ : undefined },     // PROD: only safe read-only tests
    { name: 'webkit',   use: { ...devices['Desktop Safari'] }, dependencies: ['setup'],
      grep: isProd ? /@prod-smoke/ : undefined },
    { name: 'mobile',   use: { ...devices['Pixel 7'] }, dependencies: ['setup'], grep: /@mobile/ },
  ],
});
```

```
// package.json scripts (cross-env makes VAR=value work on Windows too)
"scripts": {
  "test:qa":   "cross-env TEST_ENV=qa playwright test",
  "test:uat":  "cross-env TEST_ENV=uat playwright test --project=chromium",
  "test:prod": "cross-env TEST_ENV=prod playwright test --project=chromium",
  "test:smoke":"cross-env TEST_ENV=qa playwright test --grep @smoke"
}
// run:  npm run test:uat     |   PowerShell: $env:TEST_ENV='uat'; npx playwright test
```

Alternative from the docs: one project per environment with its own `baseURL` and `retries` (`{ name: 'staging', use: { baseURL: 'staging.example.com' }, retries: 2 }`). Good for 2–3 fixed environments; the `TEST_ENV` approach scales better with secrets per environment.

CI: GitHub Environments, secrets, manual choice + nightly schedule

```yaml
# .github/workflows/e2e-env.yml
name: E2E by environment
on:
  workflow_dispatch:                      # manual run with a dropdown
    inputs:
      environment:
        type: choice
        options: [qa, uat, prod]
        default: qa
  schedule:
    - cron: '0 1 * * *'                   # nightly regression on QA
jobs:
  e2e:
    runs-on: ubuntu-latest
    environment: ${{ inputs.environment || 'qa' }}    # GitHub Environment: own vars, secrets, approvals
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-node@v6
        with:
          node-version: lts/*
      - run: npm ci
      - run: npx playwright install --with-deps
      - run: npx playwright test
        env:
          TEST_ENV: ${{ inputs.environment || 'qa' }}
          BASE_URL: ${{ vars.BASE_URL }}            # per-environment variable
          TEST_USER: ${{ secrets.TEST_USER }}       # per-environment secret
          TEST_PWD: ${{ secrets.TEST_PWD }}
      - uses: actions/upload-artifact@v4
        if: ${{ !cancelled() }}
        with:
          name: report-${{ inputs.environment || 'qa' }}
          path: blob-report
          retention-days: 14
```

Environment	What runs	Settings
Local / dev	What you're working on, `--ui`	`.env.qa`, retries 0, headed when debugging
QA (PR + nightly)	PR: @smoke on Chromium · Nightly: full regression, all browsers + mobile	retries 2, sharded, trace on first retry
UAT / staging	Release regression before go-live	Environment approvals in GitHub, real-like data
Production	**@prod-smoke only**: read-only checks after deployment	retries 0, dedicated read-only user, never create data

> 🧠 **"One suite, many environments: TEST_ENV picks the .env locally; GitHub Environments supply vars + secrets on CI; prod runs smoke only."**

> *"Projects separate browsers, devices and setup, and a TEST_ENV variable selects the environment. Locally it loads .env.qa or .env.uat through dotenv, with npm scripts using cross-env; on CI each GitHub Environment (qa, uat, prod) holds its own BASE_URL variable and credentials as secrets, chosen by a workflow_dispatch dropdown or the nightly schedule. Production is guarded: retries 0, a read-only user, and the project grep runs only @prod-smoke tests. The config tags runs with the environment name, so merged reports show which environment each result came from."*

20 · Test Filtering: Run the Right Tests at the Right Time

Filter by	How	Real-time use
File / folder / line	`npx playwright test payments/` · `login.spec.ts:42`	Developer working on one area
Title	`-g "transfer"` · `-G "legacy"` (invert)	Quick re-run of a feature
Tags	`--grep @smoke` · `--grep-invert @flaky` · OR `"@a|@b"` · AND `"(?=.*@a)(?=.*@b)"`	Suites: @smoke, @regression, @payments, @mobile, @prod-smoke
Project	`--project=chromium` · `--project="Mobile*"`	Browser/device/environment subset
Changed files	`--only-changed=origin/main`	Fast PR feedback
Last failures	`--last-failed`	Fix-and-rerun loop
A list of tests	`--test-list quarantine.txt` / `--test-list-invert`	Run or exclude a quarantine list
Config (permanent)	`testDir`, `testMatch`, `testIgnore`, `grep`, `grepInvert` (top level or per project)	Mobile project runs only @mobile; prod runs only @prod-smoke
In code	`test.skip(condition)`, `test.fixme`, `test.only`	Environment/browser limits, known defects

```
// config: permanent filters per project
projects: [
  { name: 'api',    testDir: './tests/api' },                                  // API tests only
  { name: 'e2e',    testDir: './tests/e2e', testIgnore: '**/*.draft.spec.ts',
                   grepInvert: /@quarantine/ },                                // hide quarantined tests
  { name: 'mobile', testDir: './tests/e2e', use: { ...devices['iPhone 14'] }, grep: /@mobile/ },
],
```

Pipeline stage	Filter used
Pull request	`--grep @smoke --project=chromium` (+ `--only-changed=origin/main`) · target < 10 min
Merge to main	`--grep @regression`, Chromium + WebKit, sharded
Nightly	Everything except `@quarantine`, all projects incl. mobile
Post-deploy prod	`TEST_ENV=prod --grep @prod-smoke`
Quarantine job	`--grep @quarantine --repeat-each=5` to confirm flaky fixes

🧠 Filter by **"Where (file/project), What (title/tag), When (changed/last-failed)"**. Tags drive the pipeline.

> *"Filtering is how we keep feedback fast: pull requests run @smoke on Chromium plus tests in changed files, merges run @regression sharded across browsers, nightly runs everything except @quarantine including mobile, and after each production deployment we run @prod-smoke against prod. Permanent rules live in config through testDir, testMatch, testIgnore and per-project grep, and quick local filters use file paths, -g, --project and --last-failed."*

21 · Reporters: Who Needs Which Report

Reporter	Output	Use for
`list`	One line per test (default locally); `printSteps`	Developers' console
`line` / `dot`	Compact; `dot` is the CI default	Short CI logs
`html`	Interactive report: steps, errors, screenshots, video, **trace**; `open: 'never' | 'on-failure' | 'always'`, `outputFolder`	QA and developers analysing failures
`junit`	XML (`outputFile`)	Jenkins / Azure DevOps test tabs, test management import
`json`	Full results as JSON (`outputFile`)	Custom dashboards, metrics (pass rate, flaky rate)
`github`	Failure annotations on the PR / workflow	Reviewers see failures inline
`blob`	Raw results zip (`blob-report/`)	**Sharding / multi-env merging** into one report
Third-party	Allure, Monocart, ReportPortal, Mail, ...	Management dashboards, history, trends

```
// playwright.config.ts : different reporters locally and on CI
reporter: process.env.CI
  ? [['blob'], ['github'], ['junit', { outputFile: 'results/junit.xml' }], ['./reporters/teams-summary.ts']]
  : [['list'], ['html', { open: 'on-failure' }]],

// CLI overrides: --reporter=line (replace) · --add-reporter=json (add) · PLAYWRIGHT_JSON_OUTPUT_NAME=out.json
```

```
// reporters/teams-summary.ts : custom reporter posting a run summary (Teams/Slack webhook)
import type { Reporter, FullResult, TestCase, TestResult } from '@playwright/test/reporter';

export default class TeamsSummary implements Reporter {
  private passed = 0; private failed = 0; private flaky = 0;

  onTestEnd(test: TestCase, result: TestResult) {
    if (result.status !== test.expectedStatus && result.retry < test.retries) return;  // wait for final attempt
    const outcome = test.outcome();                                                     // expected | unexpected | flaky |
skipped
    if (outcome === 'expected') this.passed++;
    else if (outcome === 'flaky') this.flaky++;
    else if (outcome === 'unexpected') this.failed++;
  }

  async onEnd(result: FullResult) {
    await fetch(process.env.TEAMS_WEBHOOK!, { method: 'POST', headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ text: `E2E ${process.env.TEST_ENV}: ${result.status} | passed ${this.passed}, ` +
        `failed ${this.failed}, flaky ${this.flaky} | ${process.env.REPORT_URL ?? ''}` }) });
  }
}
```

```
// attach evidence to the HTML report from a test
await test.info().attach('transfer-receipt', { body: await page.screenshot(), contentType: 'image/png' });
```

🧠 **"list for me, html for QA, junit for CI tools, github for PRs, blob for shards, custom for managers."**

> *"Locally we use list plus HTML that opens on failure. On CI we write blob reports per shard, merged into one HTML report with traces, add the github reporter for PR annotations and JUnit XML for test management, and a small custom reporter posts a pass/fail/flaky summary with the report link to our Teams channel. For leadership we publish trends with a third-party dashboard such as Allure, and tests attach evidence like transaction receipts through testInfo.attach."*

22 · Retries & Flaky Tests (Local vs CI)

Where	Setting	Why
Local	`retries: 0`	See failures immediately; don't hide flakiness from the author
CI (QA/UAT)	`retries: process.env.CI ? 2 : 0` + `trace: 'on-first-retry'`	Absorb rare environment hiccups and capture a trace for the retry
Production smoke	`retries: 0`	Any failure is a real alarm
One group	`test.describe.configure({ retries: 1 })`	Known slow third-party flow
CLI	`--retries=3`	One-off run
Release gate	`--fail-on-flaky-tests`	Flaky = fail before go-live

Outcome	Meaning
passed	Passed on the first run
flaky	Failed first, **passed on a retry**: shown separately in the report: a warning, not a success
failed	Failed on the first run and every retry

How a retry runs: after a failure the worker process is **thrown away and a new one starts** (fresh browser); `beforeAll` runs again; the failed test is re-run before moving on. In serial mode the **whole group** is retried.

```
// clean server-side state before a retry
test('book fixed deposit', async ({ page, request }, testInfo) => {
  if (testInfo.retry) {                                    // 0 on first run, 1, 2 on retries
    await request.delete(`/api/test/fd-bookings?runId=${process.env.RUN_ID}`);  // remove half-created FD
  }
  // ... book FD ...
});
```

Flaky-test policy (lead)	Action
Track	Flaky count per run from the report/JSON; trend it weekly
Investigate	Open the trace from the retry: timing, locator, data or environment?
Quarantine	Tag `@quarantine`, exclude from gates (`grepInvert`), raise a ticket with an owner and a date
Fix root cause	Web-first assertions, unique data, better locators: **never** add `waitForTimeout`
Verify	`--repeat-each=10` before removing the quarantine tag

> 🧠 **"0 local, 2 on CI, 0 in prod; trace the first retry; flaky is a bug, not a pass."**

> *"Locally retries are 0 so authors see flakiness straight away. On CI we allow 2 retries with trace on first retry, so a rare environment blip doesn't block a merge but we still get evidence; production smoke has 0 retries. Playwright reports those tests as flaky, not passed, and we track that number. Flaky tests get a trace review, a @quarantine tag excluded from the gate, an owned ticket, a root-cause fix, and --repeat-each verification. For release candidates we add --fail-on-flaky-tests, and tests use testInfo.retry to clean up half-created data before retrying."*

23 · Sharding with GitHub Actions & Merging Reports

Sharding: split one suite across **several machines** (CI jobs) that run at the same time: `--shard=1/4` runs the first quarter. Each machine still uses its own **workers**.

```
1200 tests -> shard 1/4 (job A, 4 workers) -> blob-report-1.zip -+
              shard 2/4 (job B, 4 workers) -> blob-report-2.zip -+-> merge-reports -> ONE report
              shard 3/4 (job C, 4 workers) -> blob-report-3.zip -+   (html + junit/github)
              shard 4/4 (job D, 4 workers) -> blob-report-4.zip -+
40 min on 1 machine  ~=  10-12 min with 4 shards
```

```
// playwright.config.ts
fullyParallel: true,                                  // balance by TEST, not by file
reporter: process.env.CI ? 'blob' : 'html',           // each shard writes blob-report/
```

```yaml
# .github/workflows/playwright-sharded.yml
name: Playwright Tests (sharded)
on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
jobs:
  playwright-tests:
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false                              # let all shards finish
      matrix:
        shardIndex: [1, 2, 3, 4]
        shardTotal: [4]
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-node@v6
        with:
          node-version: lts/*
      - run: npm ci
      - run: npx playwright install --with-deps
      - name: Run Playwright tests
        run: npx playwright test --shard=${{ matrix.shardIndex }}/${{ matrix.shardTotal }}
        env:
          BASE_URL: ${{ vars.BASE_URL }}
          TEST_PWD: ${{ secrets.TEST_PWD }}
      - name: Upload blob report to GitHub Actions Artifacts
        if: ${{ !cancelled() }}
        uses: actions/upload-artifact@v4
        with:
          name: blob-report-${{ matrix.shardIndex }}
          path: blob-report
          retention-days: 1

  merge-reports:
    if: ${{ !cancelled() }}                         # merge even if some shards failed
    needs: [playwright-tests]
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-node@v6
        with:
          node-version: lts/*
      - run: npm ci
      - name: Download blob reports from GitHub Actions Artifacts
        uses: actions/download-artifact@v5
        with:
          path: all-blob-reports
          pattern: blob-report-*
          merge-multiple: true                      # all zips into one folder
      - name: Merge into HTML Report
        run: npx playwright merge-reports --reporter html ./all-blob-reports
      - name: Upload HTML report
        uses: actions/upload-artifact@v4
        with:
          name: html-report--attempt-${{ github.run_attempt }}
          path: playwright-report
          retention-days: 14
```

Must-know about sharding	Detail
Balancing	With `fullyParallel` tests are split individually; without it, **whole files** go to shards (uneven if one file is huge). Skipped tests aren't counted.
Blob reporter	Names include the shard number, so they don't clash; contain everything needed to rebuild any report
merge-reports options	`--reporter=html,github` (several) · `--config=merge.config.ts` (reporter options, e.g. `[['html', { open: 'never' }]]`)
Different OS / machines	Merging blobs from different OSes needs an explicit merge config (tests root)
Multiple environments	Set `tag: process.env.CI_ENVIRONMENT_NAME` in config so merged results show the environment
Exit code	Each shard job fails on its own test failures; the merge job only builds the report
Re-run failed per shard	`--last-failed --last-failed-file=.cache/last-run-shard-1.json` (one file per shard)
How many shards?	Total time ÷ target time (e.g. 40 min ÷ 10 min = 4); watch environment load and CI minutes cost
Workers vs shards	Workers = processes on **one** machine; shards = **machines**; total parallelism = shards × workers

🧠 **"Shard the suite, blob the results, merge the report."** fullyParallel for balance · `!cancelled()` so reports always upload.

> *"Our nightly regression takes about 40 minutes on one machine, so we shard it four ways with a GitHub Actions matrix, shardIndex 1 to 4, each running npx playwright test --shard with its own workers. fullyParallel makes shards balance by test rather than by file. On CI each shard writes a blob report uploaded as an artifact even when tests fail, and a merge-reports job downloads them with merge-multiple and runs npx playwright merge-reports to produce one HTML report with traces, plus GitHub annotations. With fail-fast off, every shard finishes, and the environment tag shows which environment the results came from."*

24 · Timeouts: Every Level Explained

```
globalTimeout (whole run, default none)
  +-- test timeout (default 30 s) = test body + beforeEach/afterEach + test fixtures
        |-- actionTimeout      click/fill...      (default none -> limited by test timeout)
        |-- navigationTimeout  goto/waitForURL... (default none -> limited by test timeout)
        +-- expect timeout     web-first assertion retries (default 5 s)
  beforeAll / afterAll: own timeout = test timeout · worker fixtures: own timeout
```

Timeout	Default	Set globally / locally	Real-time use
Test	30 s	Config `timeout: 60_000` · `test.setTimeout(120_000)` · `test.slow()` (×3) · `testInfo.setTimeout()` in beforeEach · `test.describe.configure({ timeout })`	Slow UAT, long journeys (account opening)
Expect	5 s	Config `expect: { timeout: 10_000 }` · `toHaveText('Paid', { timeout: 30_000 })`	Async status updates (payment "Processing" → "Paid")
Action	none	`use: { actionTimeout: 15_000 }` · `click({ timeout: 5_000 })`	Fail fast with a clear "click timed out" instead of a vague test timeout
Navigation	none	`use: { navigationTimeout: 30_000 }` · `goto('/', { timeout: 60_000 })`	Heavy dashboards, SSO redirects
Global	none	`globalTimeout: 60 * 60 * 1000` · `--global-timeout`	Stop a hung CI run but still produce reports
Fixture	test's	`[fn, { timeout: 60_000 }]`	Slow test-data creation fixture
Hooks	test's	`test.setTimeout(60_000)` inside beforeAll	One-time heavy setup per file

```
// config: sensible defaults for a slow-ish QA environment
export default defineConfig({
  timeout: 60_000,
  globalTimeout: process.env.CI ? 60 * 60 * 1000 : undefined,
  expect: { timeout: 10_000 },
  use: { actionTimeout: 15_000, navigationTimeout: 30_000 },
});

// test-level adjustments
test('download 12-month statement', async ({ page }) => {
  test.slow();                                                    // 60 s -> 180 s
  const download = page.waitForEvent('download', { timeout: 120_000 });
  await page.getByRole('button', { name: 'Download PDF' }).click();
  expect((await download).suggestedFilename()).toMatch(/statement.*\.pdf/);
});

test('payment status becomes Paid', async ({ page }) => {
  await page.getByRole('button', { name: 'Pay' }).click();
  await expect(page.getByTestId('status')).toHaveText('Paid', { timeout: 45_000 });   // async backend
});

test.describe('Account opening journey', () => {
  test.describe.configure({ timeout: 180_000 });                  // long multi-step flow
  test('open savings account end to end', async ({ page }) => { /* ... */ });
});
```

Error you'll see	Meaning / fix
`Test timeout of 30000ms exceeded.`	Whole test too slow: find the slow step in the trace; raise only for genuinely long flows
`expect.toHaveText with timeout 5000ms` (call log)	Assertion never became true: wrong text/locator, or backend slower: raise *that* assertion's timeout
`locator.click: Timeout 15000ms exceeded`	Element never actionable (hidden, disabled, covered): check the locator/overlay
`Timed out waiting 3600s for the entire test run`	Global timeout hit: shard, or find hung tests

> **Never** use `page.waitForTimeout(5000)` as a fix: it slows every run and still flakes. Wait for a **condition** (web-first assertion, `waitForResponse`, `waitForURL`) and tune the **specific** timeout.

> 🧠 **"Global › Test › (Action · Navigation · Expect)"**. Defaults: **30 s test, 5 s expect, others none**. Raise the **narrowest** timeout that fixes the problem.

> *"Playwright has layered timeouts: a 30-second test timeout covering the body, hooks and fixtures; a 5-second expect timeout for web-first assertions; optional action and navigation timeouts; and a global timeout for the whole run. In our config we raise the test timeout to 60 seconds and expect to 10 for the QA environment, set actionTimeout to 15 seconds so failures say exactly which click hung, and set a one-hour globalTimeout on CI so hung runs still report. Long journeys use test.slow or describe.configure with a timeout, and async statuses like a payment becoming Paid get a longer timeout on that one assertion. We never use waitForTimeout; we wait for conditions."*

25 · UI Mode: The Daily Development & Debugging Cockpit

UI Mode (`npx playwright test --ui`) is an interactive app to **browse, run, watch and time-travel through tests**: a live trace viewer plus test runner in one window.

Feature	What it gives you
Sidebar & run	All test files; run everything, a file, a describe or one test (triangle icon)
Filters	By text, **@tag**, status (passed/failed/skipped) and **project**
Watch mode (eye icon)	Re-runs a test (or all) automatically when you save: write → save → see result
Timeline	Coloured strip of navigations/actions; hover for snapshots; drag a range to filter actions, console and network
Actions + Before/After	Each step with its locator and duration; the DOM **before and after** the action
Pick locator	Hover the snapshot, click an element → editable locator playground → copy into the test
Pop-out DOM	Open the snapshot in a window with browser DevTools (HTML/CSS); compare two actions side by side
Source · Call · Log	Test line (Open in VS Code) · duration, locator, strict mode · internal waits (scrolling, visible, enabled, stable)
Errors · Console · Network	Failure message with a red marker on the timeline · browser + test logs · requests with headers and bodies
Attachments · Metadata	Visual diff (expected / actual / slider) · browser, viewport, duration

```
npx playwright test --ui                                  # open UI mode
npx playwright test --ui --project=chromium               # start with one project
npx playwright test --ui-host=0.0.0.0 --ui-port=8080      # from Docker / remote VM / Codespaces (opens in a browser)
```

> **Watch out: setup projects are not run automatically** in UI mode: run the `setup` project first (filter by project) so login state exists. Binding to `0.0.0.0` exposes traces, passwords and secrets to the network: use only on trusted machines.

Tool	Best for	Where
UI Mode	Writing new tests, fixing locators, exploring a failure interactively, watch mode	Local / dev container
`--debug` (Inspector)	Stepping line by line through one test	Local
Trace Viewer	Post-mortem of a CI failure (same panels, recorded run)	Downloaded CI artifacts
VS Code extension	Run/debug from the editor, breakpoints	Local

Lead / architect use: make UI mode the **standard local workflow** in onboarding (filter by your feature tag, watch mode on, pick locators, never guess selectors); use it in **pairing sessions** and flaky-test triage (timeline + network shows timing/data problems); never in CI (CI produces traces instead).

> 🧠 **"UI mode = trace viewer that can run tests."** Filter → Watch → Pick locator → Time-travel. Run **setup** first.

> *"UI Mode is our day-to-day cockpit. Engineers open npx playwright test --ui, filter by their feature tag or project, turn on watch mode, and see each action on a timeline with before and after DOM snapshots, network, console and the internal log of auto-waits. The pick-locator playground means nobody hand-writes brittle selectors. One gotcha we document is that setup projects don't run automatically in UI mode, so you run the setup project first. For CI failures we use the trace viewer, which has the same panels for a recorded run, and in containers we expose UI mode with --ui-host and --ui-port on trusted networks only."*

26 · webServer: Start the App Before the Tests

webServer tells Playwright to **start your app (and helpers) before tests**, wait until its URL responds, and **stop it afterwards**. Used when the app runs **from the same repo** (local dev, PR checks, preview builds), not when testing an already-deployed QA/UAT environment.

Option	Meaning (defaults)
`command`	Shell command that starts the server (`npm run start`)
`url`	Playwright waits until it returns 2xx/3xx/400–403 (use instead of the deprecated `port`)
`reuseExistingServer`	Use an already running server; recommended `!process.env.CI` (reuse locally, fresh on CI)
`timeout`	Startup wait, default **60 000 ms**
`stdout` / `stderr`	`'pipe'` or `'ignore'` (defaults: stdout ignore, stderr pipe)
`cwd` · `env`	Working folder (config folder) · env vars (inherits `process.env` + `PLAYWRIGHT_TEST=1`)
`name`	Log prefix (default `[WebServer]`)
`wait`	Consider started when stdout/stderr matches a regex; named groups become env vars
`gracefulShutdown`	`{ signal: 'SIGTERM', timeout: 500 }` instead of a hard kill (ignored on Windows)
`ignoreHTTPSErrors`	Accept self-signed certs when checking `url`

```
// playwright.config.ts : front end + mock back end + OTP stub, started only for local targets
import { defineConfig } from '@playwright/test';

const BASE_URL = process.env.BASE_URL ?? 'http://localhost:3000';
const isLocal = BASE_URL.includes('localhost');

export default defineConfig({
  use: { baseURL: BASE_URL },                     // page.goto('/login') -> http://localhost:3000/login
  webServer: isLocal ? [
    {
      name: 'Frontend',
      command: process.env.CI ? 'npm run build && npm run preview -- --port 3000'   // prod-like build on CI
                              : 'npm run dev -- --port 3000',                     // hot reload locally
      url: 'http://localhost:3000',
      reuseExistingServer: !process.env.CI,
      timeout: 120_000,                           // build can take time
      stdout: 'ignore', stderr: 'pipe',
    },
    {
      name: 'MockAPI',
      command: 'npx json-server --watch mocks/db.json --port 3333',
      url: 'http://localhost:3333/accounts',
      reuseExistingServer: !process.env.CI,
      env: { ...process.env, MOCK_LATENCY_MS: '200' } as Record<string, string>,
    },
    {
      name: 'OTPStub',
      command: 'node mocks/otp-stub.js',
      url: 'http://localhost:4010/health',
      gracefulShutdown: { signal: 'SIGTERM', timeout: 500 },
    },
  ] : undefined,                                   // deployed QA/UAT: nothing to start
});
```

Architect decision	Guidance
Dev server vs build on CI	CI should test a **production build** (build + preview); dev server locally for speed
Target is deployed	Skip `webServer` (as above); point `baseURL` at QA/UAT
Many services	Array of servers for 2–3 processes; for real stacks (DB, queues) prefer **Docker Compose** started by the CI job, and keep `webServer` for the app/stubs
Slow or flaky startup	Raise `timeout`, use a dedicated `/health` URL, or `wait` on a "ready" log line
Port clashes	Fixed ports per service, `reuseExistingServer` locally, unique ports in parallel CI jobs
Logs when it fails	`stderr: 'pipe'` (and `stdout: 'pipe'` while debugging) with a clear `name`

> 🧠 **"Command starts it, url proves it, reuse locally, fresh on CI, baseURL uses it."**

> *"When the app lives in the same repository, webServer starts it before the tests and stops it after. In our config it launches the front end, a mock API and an OTP stub as an array, each with a URL Playwright polls until it's ready, a longer timeout for the build, piped stderr and a name for readable logs. Locally we reuse a running dev server; on CI reuseExistingServer is false and we start a production build. When BASE_URL points at a deployed QA or UAT environment, webServer is switched off, and heavier dependencies such as databases run through Docker Compose in the pipeline."*

27 · Best Practices: Lead & Architect Playbook

Best practice (playwright.dev)	What it means	How the team enforces it
Test user-visible behaviour	Assert what users see; not function names or CSS classes	Review checklist; role/text locators only
Isolated tests	Own storage, cookies, data; shared setup in hooks; login once via setup project	Data fixtures, storageState per role, `--shuffle` runs to catch order dependence
Don't test third parties	Mock what you don't control with `page.route()`	Mocks for credit bureau, payment gateway, SMS; separate contract tests
Control test data	Stable staging environment, known data; consistent OS/browser for visual tests	Seeded data with run IDs; visual tests in the official Docker image
Use locators (user-facing, chained, filtered)	Auto-wait + retry; `getByRole`, `filter({ hasText })`; avoid CSS/XPath chains	ESLint + review; `data-testid` contract with developers
Generate locators	Codegen / VS Code / UI mode picker	Onboarding guide; no hand-written XPath
Web-first assertions	`await expect(locator).toBeVisible()`, not `expect(await locator.isVisible())`	Lint rule `prefer-web-first-assertions`
Debug locally, trace on CI	`--debug`, VS Code locally; traces (not just videos) on CI, first retry	Config default; report artifact always uploaded
Test across browsers	Projects for Chrome, Firefox, Safari (WebKit)	Chromium on PR; all browsers nightly
Keep Playwright up to date	`npm install -D @playwright/test@latest`	Renovate/Dependabot monthly; upgrade branch runs full suite
Run on CI	Every commit/PR; Linux (cheaper); install only needed browsers	`npx playwright install chromium --with-deps` on PR jobs
Lint tests	`@typescript-eslint/no-floating-promises` (missing `await`); `tsc --noEmit`	Lint + type check as a required CI step
Parallelism & sharding	`mode: 'parallel'` / `fullyParallel`; `--shard`	Sections 17 and 23
Soft assertions	`expect.soft()` checks many things, reports all failures	Dashboards/forms with many fields

```
// Mock a third party you don't control (credit bureau score) -> deterministic test
await page.route('**/api/credit-bureau/score*', route =>
  route.fulfill({ status: 200, json: { score: 780, band: 'Excellent' } }));
await page.goto('/loans/personal/apply');
await expect(page.getByText('Pre-approved up to ₹10,00,000')).toBeVisible();

// Soft assertions: check every card on the dashboard in one run
await expect.soft(page.getByTestId('savings-balance')).toBeVisible();
await expect.soft(page.getByTestId('fd-summary')).toContainText('Fixed Deposits');
await expect.soft(page.getByTestId('loan-summary')).toContainText('Outstanding');
```

```
// eslint.config.mjs : enforce best practices automatically
import tseslint from 'typescript-eslint';
import playwright from 'eslint-plugin-playwright';

export default tseslint.config({
  files: ['tests/**/*.ts'],
  extends: [playwright.configs['flat/recommended']],
  languageOptions: { parserOptions: { project: true } },
  rules: {
    '@typescript-eslint/no-floating-promises': 'error',     // missing await
    'playwright/no-wait-for-timeout': 'error',              // no hard sleeps
    'playwright/prefer-web-first-assertions': 'error',
    'playwright/no-focused-test': 'error',                  // no test.only
    'playwright/no-force-option': 'warn',                   // { force: true } hides real issues
    'playwright/no-page-pause': 'error',
  },
});
// CI quality gate:  npx eslint tests  &&  npx tsc --noEmit  &&  npx playwright test
```

Framework architecture (architect view)

```
tests/            specs: business flows only, tags, no selectors       (login.spec.ts)
  uses
fixtures/         roles, page objects, test data, API clients, guards  (mergeTests -> one import)
  uses
pages/ + components/   POM: locators + actions per page / shared widget (header, OTP modal)
  uses
api/ + utils/     API clients, data builders, date/currency helpers
  reads
config/           playwright.config.ts, .env.*, projects, reporters, test-data files
CI                lint -> type check -> smoke (PR) -> sharded regression -> merged report
```

Anti-pattern	Do instead
`page.waitForTimeout(3000)`	Web-first assertion / `waitForResponse`
Long CSS/XPath, `nth-child`	`getByRole`, `getByLabel`, `getByTestId`, `filter`
Tests depending on each other / shared accounts	Isolated tests, data fixtures, per-worker users
Logging in through the UI in every test	Setup project + `storageState`
`{ force: true }` to "fix" clicks	Find the overlay/disabled state; assert it
Raising global timeouts to hide slowness	Fix the cause; raise the narrowest timeout
Retries to hide flaky tests	Quarantine, fix the root cause, track the flaky rate
Secrets in code / committed auth files	CI secrets, `.gitignore` for `.env.*` and `playwright/.auth`

🧠 **"User-facing, Isolated, Mocked, Awaited, Traced, Linted, Up-to-date"** (U-I-M-A-T-L-U): the 7 habits of a healthy Playwright suite.

> *"I turn Playwright's best practices into guardrails rather than a wiki page. Tests assert user-visible behaviour through role, label and test-id locators agreed with developers. Every test is isolated, with data fixtures and storageState per role, and third parties like the credit bureau or payment gateway are mocked with page.route. Web-first assertions and no hard waits are enforced by eslint-plugin-playwright and the no-floating-promises rule, and lint plus tsc --noEmit run as a required CI step before tests. CI runs on Linux with only the browsers needed, traces on first retry, Chromium on pull requests and all browsers nightly, sharded, and Renovate keeps Playwright current. Architecturally, specs contain only business flows; fixtures, page objects and API clients sit underneath, so a UI change is fixed in one place."*

Part 2 · Playwright Guides: Methods, Syntax, Project Use & Interview Lines

Each guide: **what it is → methods & syntax → how it is used in a project → "In my project I used…" → memory hook**. Examples use a banking app.

#	Guide	Remember
G1	**Library**	**@playwright/test = runner + fixtures + expect; playwright = bare browser automation.** Launch → Context → Page → Close.
G2	**Accessibility testing**	**Axe = Analyze, Assert empty, Attach.** Tags = WCAG level; include/exclude = scope; disableRules = known issues.
G3	**Actions**	**fill for forms, press for keys, pressSequentially for picky widgets, setInputFiles for uploads.** Never `force: true` to hide a real problem.
G4	**Assertions**	**await expect(LOCATOR) waits; expect(VALUE) doesn't.** poll = function, toPass = block, soft = keep going.
G5	**API testing**	**API to arrange, UI to act, API to assert.** request = fixture, toBeOK = 2xx, dispose own contexts.
G6	**Authentication**	**Log in once, save state, reuse everywhere.** Empty state = guest. .auth folder in .gitignore.
G7	**Auto-waiting**	**V-S-R-E-E: Visible, Stable, Receives events, Enabled, Editable.** Wait for conditions, never for time.
G8	**Browsers**	**3 engines + 2 brands: Chromium, Firefox, WebKit + Chrome, Edge (channel).** Install with deps on Linux.
G9	**Chrome extensions**	**Extensions need: Persistent context + Chromium + load-extension args + ID from service worker.**
G10	**Clock**	**install → fastForward/runFor → assert.** setFixedTime for 'today is…' screens.
G11	**Component testing**	**Component tests = small, fast, many; E2E = few, full journeys.** mount → interact → assert → update.
G12	**Debugging tests**	**Local = --debug / UI / VS Code; CI = trace.** pause() to stop, slowMo to watch, DEBUG=pw:api to read.
G13	**Dialogs**	**Listen first, then click; accept or dismiss, never ignore.** Default = auto-dismiss.
G14	**Downloads**	**Promise first, click second, then saveAs.** suggestedFilename to assert the name.
G15	**Evaluating JavaScript**	**evaluate runs in the browser: pass args, return serialisable values.** addInitScript = before load.
G16	**Events**	**on = always, once = next one, waitForEvent = await it.** Start waiting before the action.
G17	**Extensibility**	**Extend expect for business rules; register selectors only when built-ins can't.**
G18	**Frames**	**frameLocator(selector) → then normal locators.** Nested frame = chain frameLocator.
G19	**Handles**	**Handle = a pointer to one node (can go stale); Locator = a recipe to find it every time.** Prefer locators.
G20	**Isolation**	**Browser = expensive, shared; Context = cheap, per test; Page = a tab inside a context.**
G21	**Locators**	**Really Lazy People Take A Tiny Test** (Role, Label, Placeholder, Text, Alt, Title, TestId).
G22	**Mock APIs**	**route → fulfill (fake), continue (pass/modify), abort (block), fetch (real + tweak).** Register before navigation.
G23	**Mock browser APIs**	**addInitScript = mock before load; exposeFunction = spy back to the test.**
G24	**Navigations**	**goto to go, toHaveURL to confirm, waitForURL for redirects.** Avoid 'networkidle'; wait for something visible.
G25	**Network**	**Watch (on/waitForResponse), change (route), block (abort), auth (httpCredentials).**
G26	**Other locators**	**Other locators = last resort.** CSS > XPath; :has-text and :visible when stuck.
G27	**Pages**	**Popup: promise → click → await → assert.** Tab = page; profile = context.
G28	**Page object models**	**Locators in the page, steps in methods, asserts in tests, objects via fixtures.**
G29	**Screenshots**	**screenshot = picture; toHaveScreenshot = compare.** Mask dynamic or sensitive data; same OS for baselines.
G30	**Service Workers**	**Block them by default; test them on purpose (Chromium).**
G31	**Snapshot testing**	**Aria snapshot = structure (YAML), screenshot = pixels.** Generate with ariaSnapshot(), update with -u.
G32	**Test generator**	**Record → refine → refactor into POM.** Never commit raw recordings.
G33	**Touch events (legacy)**	**tap() for taps; dispatchEvent(touch*) for gestures.** hasTouch must be on.
G34	**Trace viewer**	**Trace = flight recorder.** on-first-retry on CI; show-trace or trace.playwright.dev to open.

G35	**Videos**	**Video = for people, trace = for engineers.** retain-on-failure saves space.

G1 · Library

Playwright **without the test runner** (`playwright` package): you launch browsers yourself. Used for scripts and tools, not test suites.

Method	Syntax / example
`chromium.launch()`	`const browser = await chromium.launch({ headless: false })`
`browser.newContext()`	`const context = await browser.newContext({ viewport: { width: 1280, height: 720 } })`
`context.newPage()`	`const page = await context.newPage()`
`page.goto() / actions`	`await page.goto('https://qa.bank.com'); await page.getByLabel('User ID').fill('C123')`
`browser.close()`	`await browser.close() // always close (try/finally)`
`install`	`npm i -D playwright + npx playwright install chromium`

```
import { chromium } from 'playwright';
(async () => {
  const browser = await chromium.launch();
  try {
    const page = await (await browser.newContext()).newPage();
    await page.goto('https://qa.bank.com/health');
    await page.screenshot({ path: 'health.png' });
  } finally { await browser.close(); }
})();
```

Used in project: Data-setup scripts (create 50 test customers through the UI), nightly health-check scripts, screenshots for release notes, using Playwright inside Jest/Mocha.

> **Interview:** *"In my project I used the Playwright library in a Node script that pre-creates test customers before UAT cycles; the test suites themselves use @playwright/test."*

> 🧠 **@playwright/test = runner + fixtures + expect; playwright = bare browser automation.** Launch → Context → Page → Close.

G2 · Accessibility testing

Automated WCAG checks with **axe-core** (`@axe-core/playwright`) plus Playwright's accessibility assertions. Catches many, not all, issues.

Method	Syntax / example
`new AxeBuilder({ page }).analyze()`	`const results = await new AxeBuilder({ page }).analyze()`
`assert`	`expect(results.violations).toEqual([])`
`.withTags([...])`	`.withTags(['wcag2a', 'wcag2aa', 'wcag21a', 'wcag21aa'])`
`.include() / .exclude()`	`.include('#transfer-form') .exclude('#third-party-chat')`
`.disableRules([...])`	`.disableRules(['duplicate-id']) // known issue, ticketed`
`attach results`	`await testInfo.attach('axe', { body: JSON.stringify(results, null, 2), contentType: 'application/json' })`
`a11y assertions`	`toHaveAccessibleName('Transfer') toHaveRole('button') toMatchAriaSnapshot()`

```
import AxeBuilder from '@axe-core/playwright';
test('transfer page has no WCAG A/AA violations', async ({ page }, testInfo) => {
  await page.goto('/transfer');
  const results = await new AxeBuilder({ page })
    .withTags(['wcag2a', 'wcag2aa', 'wcag21a', 'wcag21aa'])
    .include('#transfer-form')
    .analyze();
  await testInfo.attach('axe-results', { body: JSON.stringify(results, null, 2), contentType: 'application/json' });
  expect(results.violations).toEqual([]);
});
```

Used in project: Banking portals must meet WCAG 2.1 AA: a `makeAxeBuilder` fixture with our standard tags scans login, transfer and statement pages on every nightly run; known issues are excluded with a ticket.

> **Interview:** "*In my project I added axe scans to key journeys through a shared fixture with WCAG 2.1 AA tags, failing on new violations and attaching the full JSON to the report for the accessibility team.*"

> 🧠 **Axe = Analyze, Assert empty, Attach.** Tags = WCAG level; include/exclude = scope; disableRules = known issues.

G3 · Actions

Methods that **interact with elements** like a user. All auto-wait for the element to be actionable.

Method	Syntax / example
`fill()`	`await page.getByLabel('Amount').fill('500') // clears, then types`
`pressSequentially()`	`await page.getByLabel('OTP').pressSequentially('123456', { delay: 100 })`
`clear()`	`await page.getByLabel('Remarks').clear()`
`check() / uncheck() / setChecked()`	`await page.getByLabel('I agree').check(); await box.setChecked(false)`
`selectOption()`	`await page.getByLabel('Currency').selectOption('USD') // value or { label: 'US Dollar' } or ['A','B']`
`click()`	`await btn.click() | click({ button: 'right' }) | click({ modifiers: ['Shift'] }) | click({ position: { x: 5, y: 5 } })`
`dblclick() / hover() / focus() / blur()`	`await row.dblclick(); await menu.hover(); await input.focus()`
`press()`	`await page.getByLabel('Search').press('Enter') | press('Control+A')`
`setInputFiles()`	`await upload.setInputFiles('docs/pan.pdf') | (['a.pdf','b.pdf']) | ([]) to clear | ({ name, mimeType, buffer })`
`dragTo()`	`await page.getByText('Card 1').dragTo(page.getByText('Priority'))`
`mouse / keyboard`	`await page.mouse.move(100, 200); await page.mouse.down(); await page.keyboard.type('hi')`
`scrollIntoViewIfNeeded()`	`await page.getByText('Terms').scrollIntoViewIfNeeded()`

Used in project: Fund transfer forms (fill, selectOption), KYC document upload (setInputFiles with PDFs), OTP fields that need key-by-key typing (pressSequentially), context menus in admin grids (right click).

> **Interview:** "*In my project I used fill and selectOption for transfer forms, setInputFiles for KYC uploads, and pressSequentially for an OTP widget that listens to individual keystrokes.*"

> 🧠 **fill for forms, press for keys, pressSequentially for picky widgets, setInputFiles for uploads.** Never `force: true` to hide a real problem.

G4 · Assertions

`expect` checks. **Web-first** ones (`await expect(locator)...`) retry until true; generic ones check a value once. Full list: Section 5.

Method	Syntax / example
web-first	`await expect(page.getByTestId('status')).toHaveText('Paid')`
negate	`await expect(page.getByRole('alert')).not.toBeVisible()`
soft	`await expect.soft(fee).toHaveText('₹0')`
custom message	`await expect(balance, 'balance after transfer').toHaveText('₹4,500')`
expect.poll()	`await expect.poll(async () => (await request.get('/api/txn/1')).status()).toBe(200)`
toPass()	`await expect(async () => { ... }).toPass({ intervals: [1_000, 2_000], timeout: 30_000 })`
expect.configure()	`const slowExpect = expect.configure({ timeout: 30_000 })`
expect.extend()	`expect.extend({ async toHaveAmount(locator, amount) { ... return { pass, message } } })`

Used in project: Status changes after async processing (poll/longer timeout), dashboards with many fields (soft), custom `toHaveAmount` matcher that normalises ₹ and commas.

> **Interview:** *"In my project I used web-first assertions everywhere, expect.poll to wait for back-end transaction status, soft assertions for dashboard cards, and a custom toHaveAmount matcher for currency formatting."*

> 🧠 **await expect(LOCATOR) waits; expect(VALUE) doesn't.** poll = function, toPass = block, soft = keep going.

G5 · API testing

Send HTTP requests from tests with the **`request` fixture** (APIRequestContext): pure API tests, or API calls to set up/verify UI tests.

Method	Syntax / example
request fixture	`test('get account', async ({ request }) => { ... })`
get / post / put / patch / delete	`await request.post('/api/payees', { data: { name: 'Ravi' } })`
options	`{ params: { page: 1 }, headers: { Authorization: `Bearer ${t}` }, form: {...}, multipart: {...}, timeout }`
response	`res.ok() res.status() await res.json() await res.text() res.headers()`
assert	`await expect(res).toBeOK(); expect(await res.json()).toMatchObject({ status: 'ACTIVE' })`
config	`use: { baseURL: 'https://api.qa.bank.com', extraHTTPHeaders: { 'x-api-key': process.env.API_KEY! } }`
own context	`const api = await playwright.request.newContext({ baseURL }); ... await api.dispose()`
share UI cookies	`await page.request.get('/api/profile') // uses the page's logged-in cookies`

```
test('UI transfer is recorded in the API', async ({ page, request }) => {
  const payee = await (await request.post('/api/payees', { data: { name: `P_${Date.now()}` } })).json();
  await page.goto('/transfer');
  await page.getByLabel('Payee').selectOption(payee.name);
  await page.getByLabel('Amount').fill('500');
  await page.getByRole('button', { name: 'Confirm' }).click();
  const txns = await request.get('/api/transactions', { params: { payeeId: payee.id } });
  await expect(txns).toBeOK();
  expect(await txns.json()).toContainEqual(expect.objectContaining({ amount: 500 }));
});
```

Used in project: Create payees/accounts through the API before UI tests (fast, stable), verify the UI result through the API, and contract-style checks of status codes and payloads.

> **Interview:** "*In my project I used the request fixture to seed data and to cross-check UI transfers against the transactions API, plus a separate API project for back-end regression with its own baseURL and headers.*"

> 🧠 **API to arrange, UI to act, API to assert.** request = fixture, toBeOK = 2xx, dispose own contexts.

G6 · Authentication

Log in **once** and reuse the session (`storageState` = cookies + localStorage, optionally IndexedDB) instead of logging in through the UI in every test. Details: Sections 12, 15.

Method	Syntax / example
save state	`await page.context().storageState({ path: 'playwright/.auth/user.json' })`
with IndexedDB	`await page.context().storageState({ path, indexedDB: true })`
use for project	`use: { storageState: 'playwright/.auth/user.json' }, dependencies: ['setup']`
use per file/group	`test.use({ storageState: 'playwright/.auth/admin.json' })`
logged-out test	`test.use({ storageState: { cookies: [], origins: [] } })`
two roles in one test	`const ctx = await browser.newContext({ storageState: 'playwright/.auth/checker.json' })`
API login	`await request.post('/api/login', { form: {...} }); await request.storageState({ path })`
sessionStorage	`not saved: restore with context.addInitScript(...) if the app needs it`

Used in project: Setup project logs in per role (customer, maker, checker, admin), tests start authenticated; API login when the UI login has CAPTCHA/OTP in lower environments.

> **Interview:** "*In my project I used a setup project that logs in once per role and saves storageState, so 800 tests skip the login screen, and maker-checker tests open a second context with the checker's state.*"

> 🧠 **Log in once, save state, reuse everywhere.** Empty state = guest. .auth folder in .gitignore.

G7 · Auto-waiting

Before each action Playwright runs **actionability checks** and waits until they pass (or the timeout).

Method	Syntax / example
Visible	non-empty box, no visibility:hidden → click, fill, check, hover, tap, selectOption…
Stable	not animating (same box for two frames) → click, check, hover, tap, dragTo
Receives events	not covered by another element (overlay/spinner) → click, check, hover, tap
Enabled	not disabled → click, fill, check, selectOption, press…
Editable	enabled and not readonly → fill, clear, pressSequentially
force option	click({ force: true }) skips non-essential checks: last resort
explicit waits	await locator.waitFor({ state: 'visible' | 'hidden' | 'attached' | 'detached' })
page waits	page.waitForURL('**/dashboard') page.waitForResponse(...) page.waitForLoadState('domcontentloaded') page.waitForFunction(...)

Used in project: Spinners and overlays on transfer submit, disabled 'Confirm' until OTP is valid, animated side menus: no sleeps needed; `waitForResponse` when the UI gives no visible signal.

> **Interview:** "*In my project auto-waiting removed all hard sleeps: actions wait for visible, stable, enabled elements not covered by the loading overlay, and the few remaining waits are condition-based such as waitForResponse.*"

> 🧠 **V-S-R-E-E: Visible, Stable, Receives events, Enabled, Editable.** Wait for conditions, never for time.

G8 · Browsers

Playwright ships its own **Chromium, Firefox and WebKit** builds and can also drive branded **Chrome and Edge** via channels.

Method	Syntax / example
install	npx playwright install | install chromium | install --with-deps | install chrome msedge
projects	{ name: 'firefox', use: { ...devices['Desktop Firefox'] } }
branded channel	use: { channel: 'msedge' } // or 'chrome', 'chrome-beta'
browserName fixture	test.skip(browserName === 'webkit', 'not supported')
headless shell	npx playwright install --only-shell // smaller CI images
browser location	PLAYWRIGHT_BROWSERS_PATH=./pw-browsers npx playwright install
download via proxy	HTTPS_PROXY=http://proxy.bank.local:8080 npx playwright install
launch options	use: { launchOptions: { slowMo: 200, args: ['--start-maximized'] } }

Used in project: Bank users mostly use Chrome and Edge, so Edge via `channel: 'msedge'`; WebKit stands in for Safari/iPhone; browsers installed through the corporate proxy or baked into a Docker image.

> **Interview:** "*In my project we ran Chromium on every PR, plus branded Edge, Firefox and WebKit nightly; browsers were cached in a Docker image because CI agents had no internet access.*"

> 🧠 **3 engines + 2 brands: Chromium, Firefox, WebKit + Chrome, Edge (channel).** Install with deps on Linux.

G9 · Chrome extensions

Test a Chrome extension by loading it into Chromium with a **persistent context** (only way extensions load).

Method	Syntax / example
launch	`chromium.launchPersistentContext('', { channel: 'chromium', args: [`--disable-extensions-except=${p}`, `--load-extension=${p}`] })`
extension id	`let [sw] = context.serviceWorkers(); if (!sw) sw = await context.waitForEvent('serviceworker'); const id = sw.url().split('/')[2]`
open popup	`await page.goto(`chrome-extension://${extensionId}/popup.html`)`
fixtures	context + extensionId fixtures via test.extend (persistent context per test)
limits	Chromium only; channel 'chromium' for headless; branded Chrome/Edge removed side-loading flags

Used in project: A bank's browser extension for secure-login or a corporate password tool: verify the popup and that it injects the expected banner on the portal.

> **Interview:** *"In my project I tested an internal Chrome extension using a launchPersistentContext fixture with the extension loaded, reading the extension ID from its service worker and opening the popup URL directly."*

> 🧠 **Extensions need: Persistent context + Chromium + load-extension args + ID from service worker.**

G10 · Clock

Control **time** in the page (`Date`, timers): test session timeouts, countdowns and date-dependent UI without waiting.

Method	Syntax / example
setFixedTime()	`await page.clock.setFixedTime(new Date('2026-03-31T10:00:00')) // Date.now fixed, timers run`
install()	`await page.clock.install({ time: new Date('2026-03-31T08:00:00') }) // call first`
fastForward()	`await page.clock.fastForward('05:00') // jump 5 min; due timers fire once`
runFor()	`await page.clock.runFor(2000) // advance 2 s, firing all timers`
pauseAt() / resume()	`await page.clock.pauseAt(new Date('2026-03-31T23:59:50')); ... await page.clock.resume()`
setSystemTime()	`await page.clock.setSystemTime(new Date(...)) // advanced cases`

```
test('logs out after 5 minutes of inactivity', async ({ page }) => {
  await page.clock.install();
  await page.goto('/dashboard');
  await page.clock.fastForward('05:00');
  await expect(page.getByText('You have been logged out due to inactivity')).toBeVisible();
});
```

Used in project: Session-timeout warning after 5 minutes idle, OTP 30-second countdown, cut-off times (NEFT after 6 pm), month-end / leap-year statement dates.

> **Interview:** *"In my project I used page.clock to test the inactivity logout: install the clock, fast-forward five minutes and assert the logout message, instead of waiting five real minutes."*

> 🧠 **install → fastForward/runFor → assert.** setFixedTime for 'today is...' screens.

G11 · Component testing

Test UI components in isolation in a real browser. **Current docs**: a stable built-in `mount` fixture that loads stories from your dev server's component gallery; the old `@playwright/experimental-ct-*` packages are no longer published (stay on 1.62 until migrated).

Method	Syntax / example
mount()	const component = await mount('components/Expandable/Stateful')
props	await mount<typeof WithTitle>('Button/WithTitle', { title: 'Hello' })
interact/assert	await component.getByRole('button').click(); await expect(component.getByTestId('expanded')).toHaveValue('true')
update()	await component.update({ value: 2 }) // new props, state kept
config	a 'components' project with baseURL of the gallery + webServer
run	npx playwright test --project=components
legacy (older projects)	@playwright/experimental-ct-react: mount(<Button title='Hi' />)

Used in project: Design-system components (amount input with currency formatting, date picker, OTP box) tested in isolation, faster than full E2E.

> **Interview:** *"In my project front-end developers own component tests for shared widgets like the amount input and OTP box, while QA focuses on E2E journeys; I review that component and E2E coverage don't overlap."*

> 🧠 **Component tests = small, fast, many; E2E = few, full journeys.** mount → interact → assert → update.

G12 · Debugging tests

Tools to see **why** a test fails, locally and on CI.

Method	Syntax / example
Inspector	npx playwright test login.spec.ts:12 --debug
pause in code	await page.pause()
env var	PWDEBUG=1 npx playwright test // Inspector, headed, no timeout
UI mode	npx playwright test --ui // time travel + watch
VS Code	breakpoints, 'Show browser', 'Pick locator', debug from the gutter
slow motion	use: { headless: false, launchOptions: { slowMo: 500 } }
verbose API logs	DEBUG=pw:api npx playwright test
browser logs	page.on('console', m => console.log(m.text())); page.on('pageerror', e => console.log(e))
CI	trace: 'on-first-retry' → show-report → trace viewer

Used in project: Local: VS Code breakpoints or UI mode; flaky CI failures: trace from the first retry; strange waits: DEBUG=pw:api log.

> **Interview:** *"In my project I debug locally with UI mode or --debug at a specific line, and CI failures from traces recorded on the first retry, which show DOM, network and console at the failing step."*

> 🧠 **Local = --debug / UI / VS Code; CI = trace.** pause() to stop, slowMo to watch, DEBUG=pw:api to read.

G13 · Dialogs

Native browser dialogs: `alert`, `confirm`, `prompt`, `beforeunload`. By default Playwright **auto-dismisses** them.

Method	Syntax / example
handle once	`page.once('dialog', d => d.accept())`
handle all	`page.on('dialog', async d => { console.log(d.message()); await d.dismiss(); })`
dialog API	`d.type() ('alert' | 'confirm' | 'prompt' | 'beforeunload') d.message() d.defaultValue()`
accept / dismiss	`await d.accept() | await d.accept('reason text') (prompt) | await d.dismiss()`
beforeunload	`await page.close({ runBeforeUnload: true }) // dialog event fires`
rule	`register the handler BEFORE the action; a listener must accept/dismiss or the action hangs`

```
test('cancel standing instruction asks for confirmation', async ({ page }) => {
  page.once('dialog', async dialog => {
    expect(dialog.type()).toBe('confirm');
    expect(dialog.message()).toContain('cancel this standing instruction');
    await dialog.accept();
  });
  await page.getByRole('button', { name: 'Cancel SI' }).click();
  await expect(page.getByText('Standing instruction cancelled')).toBeVisible();
});
```

Used in project: 'Are you sure you want to cancel this standing instruction?' confirm, prompt for a cancellation reason, 'unsaved changes' beforeunload on the KYC form.

> **Interview:** *"In my project I used page.once('dialog') before clicking Cancel to assert the confirm message and accept it, and a separate test dismisses it to check nothing was cancelled."*

> 🧠 **Listen first, then click; accept or dismiss, never ignore.** Default = auto-dismiss.

G14 · Downloads

Catch files the page downloads, then inspect or save them.

Method	Syntax / example
wait for download	`const downloadPromise = page.waitForEvent('download') // BEFORE the click`
trigger	`await page.getByRole('button', { name: 'Download PDF' }).click()`
get it	`const download = await downloadPromise`
file name / url	`download.suggestedFilename() download.url()`
save	`await download.saveAs(testInfo.outputPath(download.suggestedFilename()))`
temp path / failure	`await download.path() await download.failure()`
config	`use: { acceptDownloads: true } // default true`

```
test('download CSV statement', async ({ page }, testInfo) => {
  await page.goto('/statements');
  const downloadPromise = page.waitForEvent('download');
  await page.getByRole('button', { name: 'Download CSV' }).click();
  const download = await downloadPromise;
  expect(download.suggestedFilename()).toMatch(/^statement_\d{8}\.csv$/);
  const file = testInfo.outputPath(download.suggestedFilename());
  await download.saveAs(file);
  expect(fs.readFileSync(file, 'utf-8')).toContain('Opening Balance');
});
```

Used in project: Account statements (PDF/CSV/Excel), interest certificates, transaction receipts: check the file name pattern, size and content (parse CSV / PDF text).

> **Interview:** *"In my project I validated statement downloads by waiting for the download event before clicking, checking the file name pattern, saving it to the test output folder and parsing the CSV rows against the on-screen transactions."*

> 🧠 **Promise first, click second, then saveAs.** suggestedFilename to assert the name.

G15 · Evaluating JavaScript

Run JavaScript **inside the page** and get the result back. The function runs in the browser: pass values as arguments (no access to test variables).

Method	Syntax / example
`page.evaluate()`	`const title = await page.evaluate(() => document.title)`
`with arguments`	`await page.evaluate(([a, b]) => a + b, [1, 2])`
`read storage`	`await page.evaluate(() => localStorage.getItem('sessionTimeout'))`
`locator.evaluate()`	`await page.getByTestId('grid').evaluate(el => el.scrollTop = el.scrollHeight)`
`locator.evaluateAll()`	`await page.getByRole('row').evaluateAll(rows => rows.length)`
`evaluateHandle()`	`const winHandle = await page.evaluateHandle(() => window)`
`before page scripts`	`await page.addInitScript(() => { window.__TEST__ = true })`
`call test code from page`	`await page.exposeFunction('sha256', (s: string) => hash(s))`

Used in project: Read feature flags/session values from localStorage, scroll virtualised grids, set a test flag before the app loads, read values not shown in the UI.

> **Interview:** *"In my project I used page.evaluate to read the session-timeout value from localStorage and addInitScript to set a test flag that disables an onboarding tour before the app loads."*

> 🧠 **evaluate runs in the browser: pass args, return serialisable values.** addInitScript = before load.

G16 · Events

Subscribe to things that **happen** in the page/context: requests, responses, console, errors, dialogs, downloads, popups.

Method	Syntax / example
`subscribe`	`page.on('console', msg => ...) page.on('pageerror', err => ...)`
`one-time`	`page.once('dialog', d => d.accept())`
`unsubscribe`	`page.off('request', listener) // or removeListener`
`wait for one event`	`const popup = await page.waitForEvent('popup') // start before the action`
`network events`	`page.on('request') page.on('response') page.on('requestfailed')`
`context events`	`context.on('page', p => ...) context.waitForEvent('page')`
`helpers`	`page.waitForRequest('**/api/transfer') page.waitForResponse(r => r.url().includes('/otp'))`

Used in project: Fail tests on JavaScript errors (`pageerror`), log failed API calls, catch new tabs, wait for the transfer API response before asserting.

> **Interview:** *"In my project an automatic fixture listens to pageerror and requestfailed events and attaches them to the report, and tests use waitForResponse when the UI gives no visible signal."*

> 🧠 **on = always, once = next one, waitForEvent = await it.** Start waiting before the action.

G17 · Extensibility

Extend Playwright: **custom selector engines** and **custom expect matchers**.

Method	Syntax / example
register engine	`await selectors.register('tag', createTagNameEngine) // before creating pages`
engine shape	`({ query(root, sel) { ... }, queryAll(root, sel) { ... } })`
use it	`page.locator('tag=button')`
isolated engine	`selectors.register('name', engine, { contentScript: true })`
custom matcher	`export const expect = baseExpect.extend({ async toHaveAmount(locator, expected) { ... } })`
test-id attribute	`selectors.setTestIdAttribute('data-qa') // or use: { testIdAttribute }`

```
import { expect as baseExpect, type Locator } from '@playwright/test';
export const expect = baseExpect.extend({
  async toHaveAmount(locator: Locator, expected: number) {
    const text = (await locator.textContent()) ?? '';
    const actual = Number(text.replace(/[₹,\s]/g, ''));
    return { pass: actual === expected, name: 'toHaveAmount', expected, actual,
             message: () => `expected amount ${expected}, got ${actual} ("${text}")` };
  },
});
// await expect(page.getByTestId('balance')).toHaveAmount(450000);
```

Used in project: A custom matcher `toHaveAmount` (₹ and Indian comma formats), `data-qa` as the team's test-id attribute; custom selector engines are rare (legacy widget libraries).

> **Interview:** *"In my project I added a custom expect matcher for currency amounts and configured data-qa as the test-id attribute; we avoided custom selector engines because built-in locators covered our needs."*

> 🧠 **Extend expect for business rules; register selectors only when built-ins can't.**

G18 · Frames

Work with content inside **iframes** (payment gateways, 3-D Secure, chat widgets, embedded reports).

Method	Syntax / example
frameLocator()	`page.frameLocator('#payment-frame').getByLabel('Card number').fill('4111…')`
contentFrame()	`page.locator('iframe[name="pg"]').contentFrame().getByRole('button', { name: 'Pay' })`
nested	`page.frameLocator('#outer').frameLocator('#inner').getByText('OK')`
Frame objects	`page.frame({ name: 'pg' }) page.frame({ url: /secure3d/ }) page.frames()`
main frame	`page.mainFrame()`
assert inside	`await expect(page.frameLocator('#pg').getByText('Payment successful')).toBeVisible()`

Used in project: Card payment page in a gateway iframe, 3-D Secure OTP iframe, an embedded Power BI report, a chatbot widget.

> **Interview:** *"In my project the payment gateway and 3-D Secure screens were iframes, so I used frameLocator chains in a PaymentGatewayPage object; locators inside frames auto-wait just like normal ones."*

> 🧠 **frameLocator(selector) → then normal locators.** Nested frame = chain frameLocator.

G19 · Handles

ElementHandle / JSHandle point to one specific DOM node or JS object. Older API: **locators are recommended** (handles can go stale and don't auto-wait or retry).

Method	Syntax / example
element handle (discouraged)	`const el = await page.$('#balance') page.$$('tr')`
from locator	`const handle = await locator.elementHandle()`
JS handle	`const winHandle = await page.evaluateHandle(() => window)`
use handle	`await handle.evaluate(el => el.getBoundingClientRect())`
dispose	`await handle.dispose()`
pass to evaluate	`await page.evaluate(([el]) => el.click(), [handle])`

Used in project: Mostly legacy code migration; occasionally passing a JS object handle into evaluate.

> **Interview:** *"In my project I replaced page.$ and ElementHandle code from an old framework with locators, because handles point to a fixed DOM node that goes stale after re-renders, while locators re-query and auto-wait."*

> 🧠 **Handle = a pointer to one node (can go stale); Locator = a recipe to find it every time.** Prefer locators.

G20 · Isolation

Each test gets a fresh **BrowserContext**: an incognito-like profile with its own cookies, storage and cache, cheap to create (milliseconds).

Method	Syntax / example
default	`test('...', async ({ page, context }) => { ... }) // new context per test`
new context	`const ctx = await browser.newContext({ storageState, locale: 'ar-AE' })`
two users in one test	`const userA = await browser.newContext(); const userB = await browser.newContext()`
context features	`ctx.cookies() ctx.addCookies([...]) ctx.clearCookies()` `ctx.grantPermissions(['geolocation'])`
close	`await ctx.close() // contexts you create yourself`

Used in project: Every test starts clean (no leftover sessions/cart), maker and checker in the same test, two customers in a P2P transfer.

> **Interview:** *"In my project isolation meant no test depended on another's session; for maker-checker flows I opened two contexts in one test, each with its own storageState."*

> 🧠 **Browser = expensive, shared; Context = cheap, per test; Page = a tab inside a context.**

G21 · Locators

The way to find elements: lazy, strict, auto-waiting. Full reference: Section 4.

Method	Syntax / example
`getByRole()`	`page.getByRole('button', { name: 'Transfer', exact: true })`
`role options`	`{ name, exact, checked, disabled, expanded, level, pressed, selected, includeHidden }`
`getByLabel / Placeholder / Text / AltText / Title / TestId`	`page.getByLabel('Amount') page.getByTestId('balance')`
`filter()`	`page.getByRole('row').filter({ hasText: 'Ravi' }).getByRole('button', { name: 'Pay' })`
`and() / or()`	`page.getByRole('button').and(page.getByTitle('Submit'))`
`nth / first / last`	`rows.first() rows.nth(2) // avoid if possible`
`count / all`	`await rows.count() for (const r of await rows.all()) { ... }`
`describe()`	`page.getByTestId('btn-77').describe('Confirm transfer button') // readable in reports`
`visible only`	`locator.filter({ visible: true })`

Used in project: Page objects hold role/label/test-id locators; tables use row filters; `data-testid` agreed with developers.

> **Interview:** "*In my project locators follow role, then label, then test-id; table rows are found with filter by text and chained to the action button, and locators live in page objects.*"

> 🧠 **Really Lazy People Take A Tiny Test** (Role, Label, Placeholder, Text, Alt, Title, TestId).

G22 · Mock APIs

Intercept network calls and **return fake responses**, modify real ones, or block them.

Method	Syntax / example
`route + fulfill`	`await page.route('**/api/rates', route => route.fulfill({ status: 200, json: { fd: 7.1 } }))`
`error / slow`	`route.fulfill({ status: 500, body: 'Server error' }) | await new Promise(r => setTimeout(r, 3000)); route.continue()`
`modify response`	`const response = await route.fetch(); const json = await response.json(); json.limit = 0; await route.fulfill({ response, json })`
`modify request`	`route.continue({ headers: { ...route.request().headers(), 'x-test': '1' } })`
`block`	`await page.route('**/analytics/**', route => route.abort())`
`all pages`	`await context.route('**/api/**', handler)`
`remove`	`await page.unroute('**/api/rates')`
`HAR record / replay`	`await page.routeFromHAR('hars/rates.har', { url: '**/api/**', update: true }) // update:false to replay`

```
test('shows friendly error when rates API fails', async ({ page }) => {
  await page.route('**/api/fd-rates', route => route.fulfill({ status: 503, json: { error: 'down' } }));
  await page.goto('/deposits/fixed');
  await expect(page.getByRole('alert')).toHaveText('Rates are temporarily unavailable. Please try later.');
});
```

Used in project: Edge cases the back end can't easily produce: 500 errors, timeouts, zero balance, limit exceeded, empty statements; third-party credit-bureau responses; recorded HARs for stable demo data.

> **Interview:** "*In my project I mocked the rates and credit-bureau APIs with page.route and fulfill to test error banners, slow responses and limit edge cases, and used route.fetch to tweak one field of a real response.*"

🧠 **route → fulfill (fake), continue (pass/modify), abort (block), fetch (real + tweak).** Register before navigation.

G23 · Mock browser APIs

Replace browser APIs the page uses (battery, geolocation details, clipboard, feature APIs) **before the page loads**.

Method	Syntax / example
`addInitScript()`	`await page.addInitScript(() => { window.navigator.getBattery = async () => mockBattery })`
`read-only API`	`Object.defineProperty(Object.getPrototypeOf(navigator), 'cookieEnabled', { value: false })`
`verify calls`	`await page.exposeFunction('logCall', (m: string) => log.push(m)) // page calls window.logCall(...)`
`change mock later`	`await page.evaluate(() => window.mockBattery._setLevel(0.2))`
`assert log`	`expect(log).toEqual(['getBattery', 'addEventListener:levelchange'])`

Used in project: Simulate cookies disabled (show warning), WebAuthn/biometric availability flags, `navigator.onLine`, device APIs the app checks before enabling features.

> **Interview:** "*In my project I used addInitScript to make navigator.cookieEnabled false and assert the 'enable cookies' banner on the login page, and exposeFunction to verify which browser APIs the page called.*"

🧠 **addInitScript = mock before load; exposeFunction = spy back to the test.**

G24 · Navigations

Moving between pages and waiting for them to load.

Method	Syntax / example
`goto()`	`await page.goto('/login') // waits for 'load' by default`
`waitUntil`	`page.goto(url, { waitUntil: 'domcontentloaded' | 'load' | 'networkidle' | 'commit', timeout: 60_000 })`
`after a click`	`await page.getByRole('link', { name: 'Statements' }).click(); await expect(page).toHaveURL(/statements/)`
`waitForURL()`	`await page.waitForURL('**/dashboard')`
`history`	`await page.reload(); await page.goBack(); await page.goForward()`
`load state`	`await page.waitForLoadState('domcontentloaded')`
`response`	`const res = await page.goto('/login'); expect(res?.status()).toBe(200)`

Used in project: SSO redirects to dashboard (waitForURL), back-button behaviour after logout (goBack must not show account data), deep links.

> **Interview:** "*In my project I navigate with relative goto via baseURL, wait for SSO redirects with waitForURL, and test that pressing Back after logout doesn't reveal account pages.*"

🧠 **goto to go, toHaveURL to confirm, waitForURL for redirects.** Avoid 'networkidle'; wait for something visible.

G25 · Network

Observe and control **all HTTP(S) traffic**: auth, proxy, monitoring, modifying, WebSockets.

Method	Syntax / example
HTTP auth	`use: { httpCredentials: { username: 'uat', password: process.env.UAT_PWD! } }`
proxy	`use: { proxy: { server: 'http://proxy.bank.local:8080' } }`
monitor	`page.on('request', r => console.log(r.method(), r.url())) page.on('response', ...)`
wait for a call	`const resp = await page.waitForResponse(r => r.url().includes('/api/transfer') && r.status() === 200)`
inspect	`resp.request().postDataJSON() await resp.json() resp.headers()`
block assets for speed	`await page.route('**/*.{png,jpg,gif}', r => r.abort())`
WebSockets	`page.on('websocket', ws => ws.on('framereceived', f => ...)) | page.routeWebSocket('wss://…/rates', ws => ...)`
offline	`await context.setOffline(true)`

```
test('transfer request sends correct payload', async ({ page }) => {
  const responsePromise = page.waitForResponse(r => r.url().includes('/api/transfers') && r.request().method() === 'POST');
  await page.getByRole('button', { name: 'Confirm' }).click();
  const response = await responsePromise;
  expect(response.request().postDataJSON()).toMatchObject({ amount: 500, currency: 'INR' });
  expect(response.status()).toBe(201);
});
```

Used in project: Verify the transfer request payload (amount, payee ID) sent by the UI, check analytics calls, basic-auth UAT, live-rate WebSocket mocking, offline banner.

> **Interview:** *"In my project I used waitForResponse with a predicate to capture the transfer API call and assert its request payload and response, and httpCredentials for the basic-auth-protected UAT."*

> 🧠 **Watch (on/waitForResponse), change (route), block (abort), auth (httpCredentials).**

G26 · Other locators

CSS, XPath and Playwright selector extras: for cases where built-in getBy* can't express it. Use sparingly.

Method	Syntax / example
CSS	`page.locator('css=button.primary') // or just page.locator('button.primary')`
XPath	`page.locator('xpath=//table//tr[td[text()="Ravi"]]') // doesn't pierce shadow DOM`
:has-text()	`page.locator('tr:has-text("Ravi")')`
:text() / :text-is()	`page.locator(':text-is("Pay")') // exact`
:visible	`page.locator('button:visible')`
:has()	`page.locator('article:has(h2:text("Loans"))')`
nth=	`page.locator('button >> nth=0')`
legacy (avoid)	`layout :right-of() / :near(), _react= / _vue= selectors`

Used in project: Legacy screens without labels/roles, complex tables, migrated Selenium XPaths during transition.

> **Interview:** *"In my project I kept CSS/XPath only for a legacy reports screen without accessible roles, and gradually replaced them with getByRole after developers added labels and test IDs."*

> 🧠 **Other locators = last resort.** CSS > XPath; :has-text and :visible when stuck.

G27 · Pages

A **Page** is a tab. Contexts can have many pages; handle new tabs and popups.

Method	Syntax / example
new tab	`const page2 = await context.newPage()`
all tabs	`context.pages()`
popup (window.open)	`const popupPromise = page.waitForEvent('popup'); await link.click(); const popup = await popupPromise`
any new page	`const pagePromise = context.waitForEvent('page'); ... const newPage = await pagePromise`
wait for it	`await popup.waitForLoadState(); await expect(popup).toHaveURL(/terms/)`
switch / close	`await page.bringToFront(); await popup.close()`
listen globally	`context.on('page', p => console.log('new tab', p.url()))`

Used in project: 'Terms & Conditions' opening in a new tab, e-statement viewer popups, 'Open in new window' links, multiple customer tabs.

> **Interview:** "*In my project the T&C link and the e-statement viewer opened new tabs, so I started waitForEvent('popup') before clicking and asserted on the popup page object.*"

> 🧠 **Popup: promise → click → await → assert.** Tab = page; profile = context.

G28 · Page object models

Class per page/component holding **locators + actions**, so tests read like business steps and UI changes are fixed in one place.

Method	Syntax / example
class	`export class LoginPage { constructor(readonly page: Page) {} }`
locators as fields	`readonly customerId = this.page.getByLabel('Customer ID')`
actions	`async login(id: string, pwd: string) { ... }`
navigation	`async goto() { await this.page.goto('/login') }`
components	`class OtpModal { constructor(readonly root: Locator) {} } // shared widget`
inject via fixture	`loginPage: async ({ page }, use) => use(new LoginPage(page))`
assertions	`keep them in tests (or expose locators/methods returning Locators)`

```
export class TransferPage {
  readonly amount: Locator;
  readonly confirm: Locator;
  constructor(readonly page: Page) {
    this.amount = page.getByLabel('Amount');
    this.confirm = page.getByRole('button', { name: 'Confirm' });
  }
  async goto() { await this.page.goto('/transfer'); }
  async transfer(payee: string, amount: string) {
    await this.page.getByLabel('Payee').selectOption(payee);
    await this.amount.fill(amount);
    await this.confirm.click();
  }
  message() { return this.page.getByTestId('transfer-message'); }
}
// test: await transferPage.transfer('Ravi Kumar', '500');
//       await expect(transferPage.message()).toHaveText('Transfer successful');
```

Used in project: pages/ and components/ folders: LoginPage, TransferPage, OtpModal, Header; fixtures provide them; tests contain no selectors.

> **Interview:** "*In my project every screen has a page object with readonly locators and business methods, shared widgets like the OTP modal are component objects, and fixtures inject them so specs read as user journeys.*"

🧠 **Locators in the page, steps in methods, asserts in tests, objects via fixtures.**

G29 · Screenshots

Capture images for evidence, debugging and **visual regression**.

Method	Syntax / example
page	`await page.screenshot({ path: 'shots/dashboard.png', fullPage: true })`
element	`await page.getByTestId('receipt').screenshot({ path: 'receipt.png' })`
buffer → report	`await testInfo.attach('receipt', { body: await page.screenshot(), contentType: 'image/png' })`
options	`{ mask: [page.getByTestId('balance')], animations: 'disabled', clip: { x, y, width, height } }`
on failure (config)	`use: { screenshot: 'only-on-failure' }`
visual assertion	`await expect(page).toHaveScreenshot('dashboard.png', { maxDiffPixels: 100, mask: [clock] })`
hide dynamic parts	`toHaveScreenshot({ stylePath: 'tests/hide-dynamic.css' })`
update baselines	`npx playwright test --update-snapshots`

Used in project: Receipts attached as audit evidence, masked balances/account numbers, visual regression of statements and the dashboard in the Docker image for consistent rendering.

> **Interview:** *"In my project screenshots on failure are automatic, receipts are attached as evidence with account numbers masked, and visual tests use toHaveScreenshot with masks, run inside the official Docker image so baselines match."*

🧠 **screenshot = picture; toHaveScreenshot = compare.** Mask dynamic or sensitive data; same OS for baselines.

G30 · Service Workers

Background scripts used by PWAs (offline, caching, push). Supported on **Chromium-based** browsers.

Method	Syntax / example
block (simplest)	`use: { serviceWorkers: 'block' }`
list	`context.serviceWorkers()`
wait for registration	`const sw = await context.waitForEvent('serviceworker')`
evaluate inside	`await sw.evaluate(() => self.registration.active?.state) // after activation`
route SW requests	`context.route('**/api/**', route => route.request().serviceWorker() ? ... : ...)`
served by SW?	`response.fromServiceWorker()`
events	`context.on('request' | 'response' | 'requestfailed') include SW requests`

Used in project: Mobile-web PWA of the bank: offline mode shows cached balances; block service workers in normal E2E so page.route mocks aren't bypassed by the SW cache.

> **Interview:** *"In my project normal E2E runs block service workers so network mocks are reliable, and a separate Chromium suite tests the PWA's offline caching with context.serviceWorkers and fromServiceWorker."*

🧠 **Block them by default; test them on purpose (Chromium).**

G31 · Snapshot testing

Aria snapshots: compare the page's accessibility tree with a YAML template: structure and text, not pixels. Also `toMatchSnapshot` for text/binary values.

Method	Syntax / example
inline	`await expect(page.getByRole('navigation')).toMatchAriaSnapshot(` - link "Accounts" - link "Transfers"`)`
file-based	`await expect(page.getByRole('main')).toMatchAriaSnapshot({ name: 'dashboard.aria.yml' })`
generate YAML	`const yaml = await page.getByRole('main').ariaSnapshot()`
regex	`- heading /Welcome, \w+/`
partial match	`- button // any button; omit names/attributes you don't care about`
children mode	`config: expect: { toMatchAriaSnapshot: { children: 'equal' } } // contain (default) | equal | deep-equal`
update	`npx playwright test -u // or --update-snapshots=changed`
value snapshot	`expect(await response.text()).toMatchSnapshot('rates.json')`

Used in project: Menu/navigation structure per role (customer vs admin), statement table headings, error-page structure: resilient to CSS changes, catches missing elements and accessibility regressions.

> **Interview:** "*In my project I used aria snapshots to lock the navigation structure for each user role and the account-summary layout; they don't break on styling changes but catch missing links or renamed headings.*"

> 🧠 **Aria snapshot = structure (YAML), screenshot = pixels.** Generate with ariaSnapshot(), update with -u.

G32 · Test generator

Codegen records your actions into test code, preferring role/text/test-id locators. Details: Sections 2 and 6.

Method	Syntax / example
start	`npx playwright codegen https://qa.bank.com`
output file / language	`npx playwright codegen -o tests/login.spec.ts --target javascript`
emulate	`--device="iPhone 14" --viewport-size=1280,720 --color-scheme=dark --lang=ar-AE`
saved login	`--save-storage=auth.json then --load-storage=auth.json`
test-id attribute	`--test-id-attribute=data-qa`
assertions	`toolbar: assert visibility / text / value`
VS Code	`'Record new' and 'Record at cursor' in the Playwright extension`

Used in project: First draft of a new journey and locator discovery for new screens; the result is refactored into page objects before commit.

> **Interview:** "*In my project codegen is used for locator discovery and first drafts, especially with --load-storage to start logged in; recordings are always refactored into page objects and fixtures.*"

> 🧠 **Record → refine → refactor into POM.** Never commit raw recordings.

G33 · Touch events (legacy)

For apps that listen to raw **TouchEvent**s (custom swipe/pinch handlers): dispatch touch events manually. Normal taps use `tap()`.

Method	Syntax / example
simple tap	`await page.getByRole('button', { name: 'Menu' }).tap() // needs hasTouch (mobile device project)`
tap by coordinates	`await page.touchscreen.tap(200, 400)`
dispatch touch	`await el.dispatchEvent('touchstart', { touches: [{ identifier: 0, clientX, clientY }], changedTouches: [...], targetTouches: [...] })`
gesture	`touchstart → several touchmove steps → touchend (pan / swipe / pinch)`
caveat	`dispatched events have isTrusted = false; disable that check in test builds if the app requires it`

Used in project: Swipe-to-dismiss notifications, card carousel swipes, pinch-zoom on the branch map in mobile web.

> **Interview:** "*In my project mobile projects use tap() for normal touches, and for the carousel's custom swipe handler I dispatched touchstart, touchmove and touchend events, then checked the result with a screenshot.*"

> 🧠 **tap() for taps; dispatchEvent(touch*) for gestures.** hasTouch must be on.

G34 · Trace viewer

A recorded, inspectable run: actions, DOM snapshots, network, console, source. Details: Section 6.

Method	Syntax / example
config	`use: { trace: 'on-first-retry' } // off | on | retain-on-failure | on-all-retries | retain-on-first-failure`
force on	`npx playwright test --trace on`
open	`npx playwright show-report → trace icon | npx playwright show-trace trace.zip | trace.playwright.dev`
manual (library)	`await context.tracing.start({ screenshots: true, snapshots: true, sources: true })`
stop + save	`await context.tracing.stop({ path: 'trace.zip' })`
group steps	`await context.tracing.group('Fill transfer form'); ... await context.tracing.groupEnd()`

Used in project: Every CI failure is analysed from the trace of the first retry; library scripts record traces manually.

> **Interview:** "*In my project traces are recorded on the first retry in CI; when a test fails I open the trace from the merged report to see the DOM, network calls and console at the failing step.*"

> 🧠 **Trace = flight recorder.** on-first-retry on CI; show-trace or trace.playwright.dev to open.

G35 · Videos

Record a video of each test's browser context.

Method	Syntax / example
config	`use: { video: 'retain-on-failure' } // off | on | retain-on-failure | on-first-retry`
with size	`use: { video: { mode: 'on-first-retry', size: { width: 1280, height: 720 } } }`
path in a test	`const path = await page.video()?.path()`
save a copy	`await page.video()?.saveAs('videos/transfer.webm') // after the page/context closes`
library	`browser.newContext({ recordVideo: { dir: 'videos/', size: { width: 1280, height: 720 } } })`
note	`video is written when the context closes; traces are usually more useful for debugging`

Used in project: Videos of failures for business stakeholders and audit evidence of UAT runs; traces for engineers.

> **Interview:** *"In my project videos are retained only on failure to save storage, mainly for sharing with business users, while engineers debug from traces."*

> 🧠 **Video = for people, trace = for engineers.** retain-on-failure saves space.